

A Project Report

On

**Multi-Target Stance Detection With Adversarial Attention  
Network**

Submitted in partial fulfillment of the requirements for the award of the degree  
of

**BACHELOR OF TECHNOLOGY**

In

**COMPUTER SCIENCE AND ENGINEERING**

By

|                               |              |
|-------------------------------|--------------|
| Ganta Greshma                 | (21NN1A0514) |
| Aradhyula Lakshmi Priya Yadav | (21NN1A0501) |
| Kavuri Meghana                | (21NN1A0531) |
| Patan Shahina Parveen         | (21NN1A0550) |
| Dundi Hari Priya              | (20NN1A05D4) |

Under the Esteemed Guidance of

Dr. M. Vasumathi Devi

Associate Professor



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**VIGNAN'S NIRULA INSTITUTE OF TECHNOLOGY AND SCIENCE FOR WOMEN**

**PEDAPALAKALURU, GUNTUR-522005**

**(Approved by AICTE, NEW DELHI and Affiliated to JNTUK, Kakinada.)**

**2021-2025**

**VIGNAN'S NIRULA INSTITUTE OF TECHNOLOGY & SCIENCE FOR  
WOMEN**

**PEDAPALAKALURU ROAD, GUNTUR-522005**

**(Approved by AICTE & Affiliated to JNTUK, Kakinada)**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**



**CERTIFICATE**

This is to certify that this project report entitled “**Multi-Target Stance Detection With Adversarial Attention Network**” is a bonafide record of work carried out by **Ganta Greshma (21NN1A0514), Aradhyula Lakshmi Priya Yadav (21NN1A0501), Kavuri Meghana (21NN1A0531), Patan Shahina Parveen (21NN1A0550), Dundi Hari Priya (20NN1A05D4)** under the guidance of and supervision of Dr. Vasumathi Devi Min partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** of **VIGNAN'S NIRULA INSTITUTE OF TECHNOLOGY & SCIENCE FOR WOMEN, GUNTUR.**

**Project Guide**

**Dr. M. Vasumathi Devi**

**Associate Professor**

**Head of the Department**

**Dr. V. Lakshman Narayana**

**Professor**

**EXTERNAL EXAMINER**

## DECLARATION

We hereby declare that the work described in this project work, entitled “**Multi-Target Stance Detection With Adversarial Attention Network**” which is submitted by us in partial fulfilment for the award of **Bachelor of Technology** in the Department of **Computer Science and Engineering** to the **Vignan’s Nirula Institute of Technology and Science for Women**, affiliated to Jawaharlal Nehru Technological University Kakinada, Andhra Pradesh, is the result of work done by us under the guidance of **Dr. M .Vasumathi Devi**, Associate Professor.

The work is original and has not been submitted for any Degree/ Diploma of this or any other university.

**Place: Guntur**

**Date:**

### PROJECT ASSOCIATES

|                               |              |
|-------------------------------|--------------|
| Ganta Greshma                 | (21NN1A0514) |
| Aradhyula Lakshmi Priya Yadav | (21NN1A0501) |
| Kavuri Meghana                | (21NN1A0531) |
| Patan Shahina Parveen         | (21NN1A0550) |
| Dundi Hari Priya              | (20NN1A05D4) |

## ACKNOWLEDGEMENT

We express our heartfelt gratitude to our beloved principal **Dr. P. Radhika** for giving a chance to study in our esteemed institution and providing us all the required resources.

We would like to thank **Dr. V. Lakshman Narayana, Associate Professor, Head of the Department of Computer science and Engineering**, for his extended and continuous support, valuable guidance and timely advices in the completion of this project thesis.

We wish to express our profound sense of sincere gratitude to our Project Guide **Dr. M. Vasumathi Devi, Associate Professor, Department of Computer Science and Engineering**, without whose help, guidance and motivation this project thesis could not have been completed the project successfully.

We also thank all the faculty of the Department of Computer Science and Engineering for their help and guidance of numerous occasions, which has given us the cogency to build-up adamant aspiration over the completion of our project thesis.

Finally, we thank one and all who directly or indirectly helped us to complete our project thesis successfully.

## PROJECT ASSOCIATES

|                               |              |
|-------------------------------|--------------|
| Ganta Greshma                 | (21NN1A0514) |
| Aradhyula Lakshmi Priya Yadav | (21NN1A0501) |
| Kavuri Meghana                | (21NN1A0531) |
| Patan Shahina Parveen         | (21NN1A0550) |
| Dundi Hari Priya              | (20NN1A05D4) |

**MULTI-TARGET STANCE DETECTION  
WITH ADVERSARIAL ATTENTION  
NETWORK**

## ABSTRACT

The "Multi-Target Adversarial Attention Network (MTAAN) for Stance Detection" is a deep learning model designed to automatically identify a speaker's stance on various topics within social media platforms. Stance detection is crucial for understanding public opinion, yet existing models such as Long Short-Term Memory (LSTM) networks and Convolutional Neural Networks (CNN) often struggle with accurately interpreting perspectives, particularly when targets are not explicitly mentioned in the text. The proposed MTAAN model addresses this challenge by integrating a pre-trained Bidirectional Encoder Representations from Transformers (BERT) encoder with an innovative adversarial attention mechanism, enhancing the model's performance in stance classification. The MTAAN architecture consists of three core modules: the Target Discriminator, Sentiment Discriminator, and Stance Detection Module. By employing an adversarial training approach, the model optimizes the stance detection module while simultaneously training the sentiment and target discriminators to minimize their classification losses. This adversarial influence significantly improves the model's ability to classify stances accurately, achieving an impressive accuracy of 97%. The MTAAN model categorizes stances into three distinct classes: Favor, Against, and Neutral. This classification enables a significant understanding of public sentiment on various issues. The key benefits of the MTAAN model include high accuracy, robust handling of unseen targets, enhanced interpretation of opinions, and real-time analysis capabilities. By leveraging deep learning and adversarial learning techniques, the MTAAN model contributes to more nuanced insights into social media discourse, making it a valuable tool for effectively gauging public opinion.

# TABLE OF CONTENTS

| <b>Chapter No</b> | <b>Name of the Chapter</b>                                           | <b>Page No</b> |
|-------------------|----------------------------------------------------------------------|----------------|
| <b>Chapter 1</b>  | <b>Introduction</b>                                                  | 1              |
| 1.1               | Introduction                                                         | 1              |
| 1.2               | Importance of Multi-Target Stance Detection                          | 2              |
| 1.3               | Overview of the Dataset                                              | 3              |
| 1.4               | Role of Deep Learning in Stance Detection                            | 3              |
| 1.5               | Limitations of Existing Models                                       | 4              |
| 1.6               | Benefits of Using Adversarial Attention Networks in Stance Detection | 4              |
| <b>Chapter 2</b>  | <b>Literature Survey</b>                                             | 6              |
| 2.1               | Literature Review                                                    | 6              |
| 2.2               | Gaps Identified from The Literature Survey                           | 11             |
| <b>Chapter 3</b>  | <b>System Analysis</b>                                               | 12             |
| 3.1               | System Analysis                                                      | 12             |
| 3.2               | Problem Definition                                                   | 12             |
| 3.3               | Existing Systems                                                     | 13             |
| 3.3.1             | Disadvantages of Existing System                                     | 13             |
| 3.4               | Proposed System                                                      | 15             |
| 3.4.1             | Advantages of Proposed System                                        | 16             |
| 3.5               | Feasibility Study                                                    | 18             |
| <b>Chapter 4</b>  | <b>Requirements Specifications</b>                                   | 21             |
| 4.1               | Purpose, Scope, Definition                                           | 21             |
| 4.1.1             | Purpose                                                              | 21             |
| 4.1.2             | Scope                                                                | 21             |
| 4.1.3             | Definition                                                           | 21             |
| 4.2               | Requirement Analysis                                                 | 22             |
| 4.2.1             | Functional Requirements                                              | 22             |
| 4.2.2             | User Requirements Analysis                                           | 23             |
| 4.2.3             | Non-Functional Requirements                                          | 23             |
| 4.3               | System Requirements                                                  | 23             |
| 4.3.1             | Software Requirements                                                | 23             |

|                                                                                   |    |
|-----------------------------------------------------------------------------------|----|
| 4.3.2 Hardware Requirements                                                       | 24 |
| <b>Chapter 5 System Design</b>                                                    | 25 |
| 5.1 System Architecture                                                           | 25 |
| 5.2 Modules                                                                       | 25 |
| 5.2.1 BERT with Multi-Target Adversarial<br>Attention Network Deep Learning Model | 26 |
| 5.2.2 Streamlit Framework with Flask for User<br>Interface                        | 30 |
| 5.3 Design Overview                                                               | 31 |
| 5.4 UML Diagrams                                                                  | 32 |
| 5.4.1 Use case Diagram                                                            | 34 |
| 5.4.2 Activity Diagram                                                            | 35 |
| 5.4.3 Sequence Diagram                                                            | 35 |
| 5.4.4 Component Diagram                                                           | 36 |
| 5.4.5 Deployment Diagram                                                          | 37 |
| 5.5 Algorithm                                                                     | 37 |
| <b>Chapter 6 Implementation</b>                                                   | 40 |
| 6.1 Steps for Implementation                                                      | 40 |
| 6.1.1 Dataset                                                                     | 40 |
| 6.1.2 Data Preprocessing                                                          | 41 |
| 6.1.3 Implementation using Python                                                 | 41 |
| 6.2 Coding                                                                        | 47 |
| <b>Chapter 7 Testing</b>                                                          | 55 |
| 7.1 Testing                                                                       | 55 |
| 7.2 Types Of Tests                                                                | 56 |
| 7.2.1 Unit Testing                                                                | 56 |
| 7.2.2 Integration Testing                                                         | 57 |
| 7.2.3 Functional Testing                                                          | 57 |
| 7.2.4 System Testing                                                              | 58 |
| 7.2.5 White Box Testing                                                           | 58 |
| 7.2.6 Black Box Testing                                                           | 58 |

|                                                           |           |
|-----------------------------------------------------------|-----------|
| 7.3 Test Cases(Steps)                                     | 59        |
| <b>Chapter 8 Results And User Interface</b>               | <b>62</b> |
| 8.1 Screenshots                                           | 62        |
| 8.2 Experimental Results                                  | 64        |
| 8.2.1 Performance of Existing and Proposed Models         | 64        |
| 8.2.2 Confusion matrix of existing and proposed models    | 67        |
| 8.2.3 Performance Metrics of existing and proposed models | 68        |
| 8.2.4 Tenfold Cross validation                            | 70        |
| <b>Chapter 9 Conclusion And FutureWork</b>                | <b>72</b> |
| 9.1 Conclusion And FutureWork                             | 72        |
| <b>Chapter 10 Bibliography</b>                            | <b>73</b> |

## LIST OF FIGURES

| <b>S.No</b> | <b>Fig No</b> | <b>Description of figure</b>                                                         | <b>Page No</b> |
|-------------|---------------|--------------------------------------------------------------------------------------|----------------|
| 1           | 1.1           | Multi-Target Adversarial Attention Network Framework                                 | 1              |
| 2           | 1.2           | Stance Classification                                                                | 3              |
| 5           | 5.1           | System Architecture                                                                  | 25             |
| 6           | 5.2           | BERT Process                                                                         | 29             |
| 7           | 5.3           | Types and categories of UML diagram                                                  | 32             |
| 8           | 5.4.1         | Use Case Diagram                                                                     | 34             |
| 9           | 5.4.2         | Activity Diagram                                                                     | 35             |
| 10          | 5.4.3         | Sequence Diagram                                                                     | 36             |
| 11          | 5.4.4         | Component Diagram                                                                    | 36             |
| 12          | 5.4.5         | Deployment Diagram                                                                   | 37             |
| 13          | 7.3.1         | User Interface                                                                       | 59             |
| 14          | 7.3.2         | Select Pre-defined Target                                                            | 59             |
| 15          | 7.3.3         | Select Customised Target                                                             | 60             |
| 16          | 7.3.4         | Give the Comment                                                                     | 60             |
| 17          | 7.3.5         | Prediction Output                                                                    | 61             |
| 18          | 8.1           | Prediction Page of Stance Detection Using Multi-Target Adversarial Attention Network | 62             |
| 19          | 8.2           | Prediction of Favor                                                                  | 63             |
| 20          | 8.3           | Prediction of Netural                                                                | 63             |
| 21          | 8.4           | Prediction of Against                                                                | 64             |
| 22          | 8.2.1         | Accuracy and Loss of different models                                                | 65             |
| 23          | 8.2.2         | Precision recall and F1-Score curve of different models                              | 66             |
| 24          | 8.2.3         | Confusion Matrix of different models                                                 | 68             |
| 25          | 8.2.4         | Performance Parameters of different Models                                           | 70             |

## LIST OF TABLES

| <b>S.No</b> | <b>Table No</b> | <b>Table Description</b> | <b>Page No</b> |
|-------------|-----------------|--------------------------|----------------|
| 1           | 2.1             | Literature Survey        | 8              |
| 2           | 6.1.1           | Dataset                  | 40             |
| 3           | 8.2.4           | 10-Fold Cross Validation | 71             |

## LIST OF ACRONYMS

| Acronyms | Definition                                                    |
|----------|---------------------------------------------------------------|
| MTAAN    | Multi-Target Adversarial<br>Attention Network                 |
| CNN      | Convolutional Neural Network                                  |
| LSTM     | Long Short-Term Memory                                        |
| NLP      | Natural Language Processing                                   |
| BERT     | Bidirectional Encoder<br>Representations from<br>Transformers |
| AT       | Adversarial Training                                          |
| TD       | Target Discriminator                                          |
| SD       | Sentiment Discriminator                                       |
| SA       | Stance Analysis                                               |
| SDM      | Stance Detection Module                                       |
| DLA      | Deep Learning and Adversarial<br>Learning                     |
| DNN      | Deep Neural Networ                                            |
| MA       | Multi-Stance Analysis                                         |
| MAE      | Mean Absolute Error                                           |
| AA       | Adversarial Attention                                         |
| MST      | Multi-Stance Training                                         |
| MSD      | Multi-Stance Detection                                        |
| MSTC     | Multi-Stance Classification                                   |
| F1       | F1 Score                                                      |
| MST-AD   | Multi-Stance Adversarial<br>Detection                         |

# **CHAPTER 1**

## **INTRODUCTION**

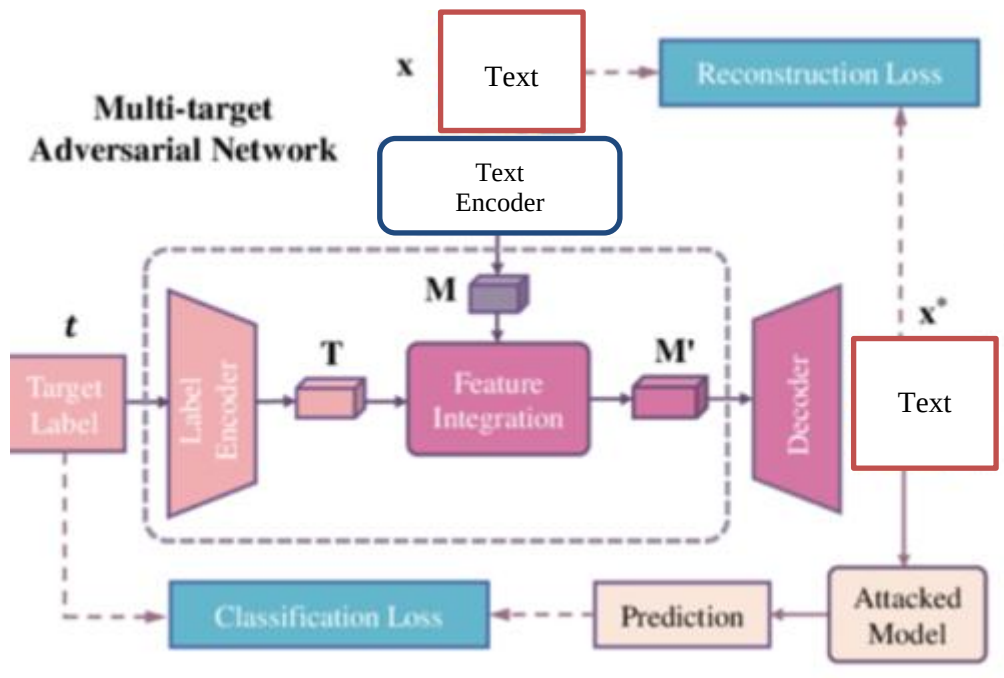
## CHAPTER 1

### INTRODUCTION

#### 1.1 Introduction

In the age of digital communication, social media has become a powerful platform where people share their views on various issues. Automatically determining how a speaker feels about a specific topic is a major challenge in Natural Language Processing (NLP). Stance detection sorts text as either supporting, opposing, or neutral regarding a particular subject. Traditional stance detection models mainly focus on one target at a time, which limits their effectiveness in real-world situations where multiple topics may be discussed at once.

A multi-target adversarial attention network is a new method for improving stance detection by using attention techniques and adversarial training. This model is designed to identify and classify stances related to several targets in the dataset and enhance its ability to generalize to new targets in the test data. Previous research has mostly focused on stance detection for a single target, and many existing models do not train jointly with both target and sentiment classifiers



**Fig 1.1 Multi-Target Adversarial Attention Network Framework**

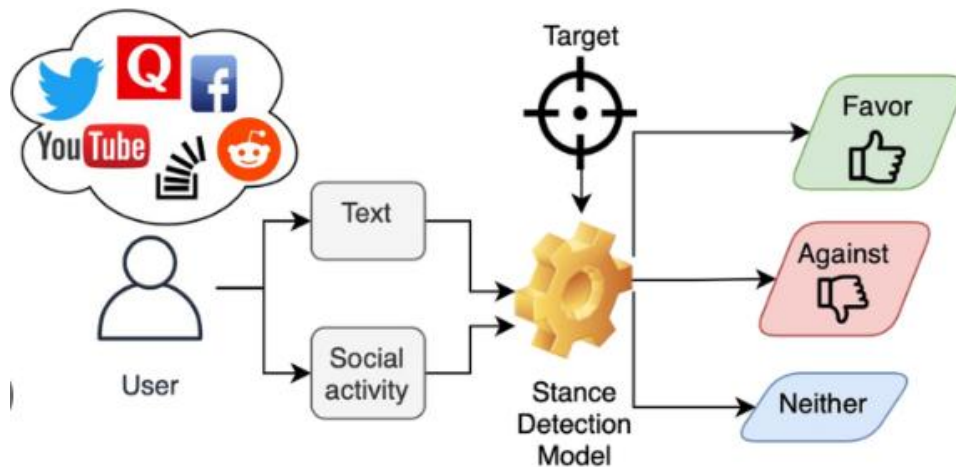
Neural network attention mechanisms are vital for stance recognition, allowing the model to focus on important parts of the input data instead of treating all sections equally. This is especially beneficial for stance detection, as relevant context is not always evenly distributed throughout the text. Furthermore, the adversarial aspect of the proposed method improves the model's ability to distinguish between different stances by training two models together: one for stance classification and another that generates challenging examples for the first model. This approach enhances the effectiveness of stance identification.

## **1.2 Importance of Multi-Target Stance Detection**

Stance detection plays a crucial role in natural language processing, particularly in applications such as fact-checking, fake news detection, and sentiment analysis. However, stance detection is often challenged by ambiguous expressions, context-dependent opinions, and diverse linguistic variations. The presence of multiple targets within a single statement further complicates accurate classification, making it essential to develop advanced computational models for precise stance determination.

Traditional stance detection methods primarily rely on rule-based approaches or classical machine learning models, which struggle with the complexity of multi-target scenarios. Manual annotation and feature engineering are time-consuming, error-prone, and fail to generalize across different datasets. Furthermore, conflicting stances in a single text increase the difficulty of accurately discerning the writer's intent, necessitating more sophisticated and adaptable solutions.

By leveraging the Multi-Target Adversarial Attention Network (MTAAN), stance detection can be significantly improved through deep learning techniques. MTAAN effectively models interactions between multiple targets by employing adversarial learning and attention mechanisms, which enhance the model's ability to distinguish fine-grained stances. This approach not only improves classification accuracy but also reduces reliance on extensive manual preprocessing, leading to more scalable and efficient stance detection systems. An advanced stance detection framework enhances information credibility, strengthens fact-checking processes, and contributes to more reliable decision-making in digital communication and media analysis.



**Fig 1.2 Stance Classification**

### 1.3 Overview of the Dataset

The dataset used for this project is the SemEval-2016 Task 6 Dataset, a well-known resource for stance detection in social media. It contains textual data from Twitter, where users express their opinions on various controversial topics. The dataset focuses on five primary targets: Feminist Movement, Hillary Clinton, Legalization of Abortion, Atheism, and Climate Change is a Real Concern.

The dataset is divided into training and test sets to ensure that the model generalizes well to unseen data. Each instance consists of a tweet labeled with a stance, which can be Favor, Against, or Neutral toward the given target. The dataset includes diverse linguistic variations, sentiment expressions, and contextual dependencies, making it suitable for real-world stance detection applications. These diverse features help train a deep learning model capable of accurately identifying and categorizing stances in social media discourse.

### 1.4 Role Deep Learning in Stance Detection

Deep learning has transformed stance detection by enabling automated, accurate, and efficient analysis of textual data. Unlike traditional methods that rely on manual annotation, deep learning models can extract contextual and semantic patterns from text, identifying stance with high precision. Transformer-based architectures, particularly advanced models like Multi-Target Adversarial Attention Network (MTAAN), enhance feature representation by focusing on critical contextual cues.

This improves classification accuracy, allowing better differentiation of stance expressions. By leveraging large-scale datasets, deep learning models learn to identify subtle variations in opinions, reducing the risk of misclassification.

The trained deep learning model is then integrated with a user-friendly interface using Streamlit. The front-end allows users to input social media text, which is then processed by the deep learning model in the backend. The model analyzes the input, applies adversarial training and attention mechanisms for enhanced feature extraction, and returns an accurate stance classification. This seamless integration ensures real-time stance detection, helping researchers and analysts track public opinion effectively.

By combining deep learning with interactive web applications, this approach makes stance detection accessible, reducing dependency on manual annotation and enabling scalable, data-driven solutions for sentiment analysis and opinion mining.

### **1.5 Limitations of Existing Models**

Traditional deep learning models such as LSTM, CNN, and Transformer-based architectures have shown effectiveness in stance detection but come with certain limitations. LSTM models struggle with capturing long-range dependencies, leading to misinterpretation of complex stance expressions. While CNN models extract local features effectively, they often fail to grasp deeper contextual relationships, reducing based accuracy in nuanced debates. Transformer models enhance contextual understanding but demand high computational resources, making them unsuitable for real-time applications. Additionally, these models lack adaptability to new targets, making stance classification less reliable in dynamic social media environments. Furthermore, most existing models function as black-box systems, limiting transparency in decision-making. To address these challenges, adversarial learning and attention mechanisms help improve contextual representation, enhancing accuracy and making stance detection more robust and interpretable.

### **1.6 Benefits of Using Adversarial Attention Networks in Stance Detection**

**Context Awareness:** By using attention-models, the system effectively captures the relationships between words, allowing for accurate stance detection even in complicated sentences.

**Improved Generalization:** The adversarial learning method makes the model stronger against noisy and misleading samples, ensuring better performance across different datasets.

**Efficient Target Differentiation:** Unlike traditional models, the proposed framework can identify stance expressions toward multiple targets at the same time, making it useful for real-world tasks like social media analysis and news sentiment classification.

**Scalability:** The system can be run on cloud platforms, allowing for large-scale stance detection across various text datasets, which helps in areas like politics, business, and social discussions.

Overall, the combination of an Adversarial Attention Network with Multi-Target Stance Detection improves accuracy, strength, and scalability, making it a valuable tool for analyzing different viewpoints in text data

.

## **CHAPTER 2**

### **LITERATURE SURVEY**

## CHAPTER 2

### LITERATURE SURVEY

#### 2.1 Literature Review

In recent years, significant research efforts have been directed toward the development of intelligent systems for accurate and efficient stance detection. Numerous researchers have explored a variety of deep learning and graph-based models to enhance the precision of stance classification across multiple targets. These studies have employed diverse datasets, feature extraction techniques, and model architectures to address challenges such as contextual ambiguity and misinformation detection. By leveraging attention mechanisms, multi-task learning, and adversarial networks, the authors have contributed to the advancement of automated stance analysis. The following section summarizes the key contributions and findings of ten notable research works in the domain of stance detection.

Li, Y. et al. [1] proposed Improved Target-Specific Stance Detection on Social Media Platforms by Delving into Conversation Threads, published in IEEE Transactions on Computational Social Systems, vol. 10, no. 6, pp. 3031-3042, Dec. 2023. This work enhances stance detection by analyzing conversation threads for better context understanding.

Chai, H. et al. [2] proposed MG-SIN: Multigraph Sparse Interaction Network for Multitask Stance Detection, published in IEEE Transactions on Neural Networks and Learning Systems, vol. 36, no. 2, pp. 3111-3125, Feb. 2025. This method utilizes a multigraph approach to improve multitask stance detection.

Li, C. et al. [3] proposed Joint Stance and Rumor Detection in Hierarchical Heterogeneous Graph, published in IEEE Transactions on Neural Networks and Learning Systems, vol. 33, no. 6, pp. 2530-2542, June 2022. This research addresses the dual challenge of stance and rumor detection using a hierarchical graph structure.

Khiabani, P. J. & Zubiaga, A. [4] proposed Few-Shot Learning for Cross-Target Stance Detection by Aggregating Multimodal Embeddings, published in IEEE Transactions on Computational Social Systems, vol. 11, no. 2, pp. 2081-2090, April 2024. This study explores few-shot learning techniques to enhance stance detection across different targets.

Sengan, S. et al. [5] proposed Fake News Detection Using Stance Extracted Multimodal Fusion-Based Hybrid Neural Network, published in IEEE Transactions on Computational Social Systems, vol. 11, no. 4, pp. 5146-5157, Aug. 2024. This approach combines multimodal data to improve fake news detection through stance analysis.

Zhang, H. et al. [6] proposed Multi-Modal Meta Multi-Task Learning for Social Media Rumor Detection, published in IEEE Transactions on Multimedia, vol. 24, pp. 1449-1459, 2022. This work focuses on leveraging multi-modal data for effective rumor detection in social media.

Zhu, Y. et al. [7] proposed A Novel Rumor Detection Method Based on Non-Consecutive Semantic Features and Comment Stance, published in IEEE Access, vol. 11, pp. 58016-58024, 2023. This research introduces a method that utilizes semantic features and comment stance for improved rumor detection.

Pandey, A. C. & Tikkiwal, V. A. [8] proposed Stance detection using improved whale optimization algorithm, published in Complex Intelligent Systems, vol. 7, pp. 1649–1672, 2021. This study applies an optimization algorithm to enhance stance detection performance.

AlturayEIF, N. et al. [9] proposed Enhancing stance detection through sequential weighted multi-task learning, published in Social Network Analysis and Mining, vol. 14, pp. 25, 2024. This work focuses on improving stance detection accuracy through a multi-task learning framework.

Gambini, M. et al. [10] proposed Evaluated large language models for user stance detection on X (Twitter), published in Machine Learning, vol. 113, pp. 7243–7266, 2024. This research assesses the effectiveness of large language models in detecting user stance on social media platforms.

Table 2.1. Literature Survey

| S.no | Title & Year of Publication                                                                                     | Work description                                                                                                                                                 | Advantages                                                                                                                                           | Limitations                                                                                                                              |
|------|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| [1]  | Improved Target-Specific Stance Detection on Social Media Platforms by Delving into Conversation Threads (2023) | The researchers developed a model called Branch-Bidirectional Encoder Representations from Transformers (BERT) to perform Conversational Stance Detection (CSD). | It enhances stance detection by considering the flow of conversation, allowing for a better understanding of opinions expressed over multiple posts. | The dataset only includes six major social media platforms in Hong Kong, which might not reflect worldwide trends or various situations. |
| [2]  | MG-SIN: Multigraph Sparse Interaction Network for Multitask Stance Detection (2025)                             | The study uses a Multigraph Sparse Interaction Network (MG-SIN) with multitask learning, task-specific and task-related graphs.                                  | Identifies both meaning and context connections between tasks, enhancing the accuracy of stance detection and sentiment analysis.                    | The model's complexity might need a lot of computing power and may not work well with all kinds of social media data.                    |
| [3]  | Joint Stance and Rumor Detection in Hierarchical Heterogeneous Graph (2022)                                     | The method combines stance and rumor detection into a single process, improving accuracy and requiring less labeled data.                                        | The method boosts accuracy by linking stance and rumor detection, needing fewer labeled examples and offering clearer insights.                      | The results may be influenced by how good the input data is and the specific social media setting.                                       |

|     |                                                                                                 |                                                                                                                                                                    |                                                                                                                                            |                                                                                                                                        |
|-----|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| [4] | Few-Shot Learning for Cross-Target Stance Detection by Aggregating Multimodal Embeddings (2024) | The CT-TN model enhances cross-target stance detection by merging text and social network features, allowing better predictions for new targets with few samples.  | This significantly outperforms existing methods in few-shot scenarios and effectively incorporates social features for better predictions. | The model's success can vary based on the quality of the input data and the type of social media platform used.                        |
| [5] | Fake News Detection Using Stance Extracted Multimodal Fusion-Based Hybrid Neural Network (2024) | This study develops Fake find, a hybrid model that leverages CNN and RNN to detect fake news by fusing multimodal features such as text, visuals, and user stance. | The model improves rumor detection by using a mix of content from different sources, resulting in more accurate outcomes.                  | The model's effectiveness can be affected by the quality of the input data and the type of social media used.                          |
| [6] | Multi-Modal Meta Multi-Task Learning for Social Media Rumor Detection (2022)                    | This paper proposed a method called Multi-modal Meta Multi-Task Learning (MM-MTL) to help detect rumors on social media.                                           | Uses both text and images for better detection. Solves problems with sharing information between different tasks                           | It doesn't mention how complex or resource-heavy the method is. The attention method's success might depend on the specific data used. |

|     |                                                                                                     |                                                                                                                                                                                   |                                                                                                                                         |                                                                                                                                                      |
|-----|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| [7] | A Novel Rumor Detection Method Based on Non-Consecutive Semantic Features and Comment Stance (2023) | This paper uses techniques like Cross-modal Transformers and graph attention networks which are useful for stance-related adversarial learning.                                   | Considers both the meaning of text and the authority of commenters. It works more effectively on well-known datasets (Weibo and PHEME). | The effectiveness on different social media platforms is not discussed. The reliance on user authority may not always be applicable in all contexts. |
| [8] | Stance Detection Using Improved Whale Optimization Algorithm (2021)                                 | This paper uses whale optimization algorithm combined with a multilayer perceptron (MLP). The whale optimization algorithm is used to update the weights and biases of the MLP    | Performs better on several standard test datasets. Provides useful insights for future research on analyzing text data.                 | The complexity of the whale optimization algorithm may require significant computational resources.                                                  |
| [9] | Enhancing Stance Detection Through Sequential Weighted Multi-Task Learning (2024)                   | It focuses on stance detection, which identifies how people feel about a specific topic. two models—Parallel Multi-Task Learning (PMTL) and Sequential Multi-Task Learning (SMTL) | Combines multiple tasks (sentiment analysis and sarcasm detection) to improve results.                                                  | Its complexity to implementing Multi-Task Learning with Transformer architectures may require significant computational                              |

|      |                                                                                  |                                                                                                                                                                                                                    |                                                                                                                                                         |                                                                                                                                                                                                             |
|------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|      |                                                                                  |                                                                                                                                                                                                                    |                                                                                                                                                         | resources.                                                                                                                                                                                                  |
| [10] | Evaluating Large Language Models for User Stance Detection on X (Twitter) (2024) | The T2S framework uses a technique called zero-shot classification with LLMs that have been trained on tasks related to understanding language. This helps it detect user opinions based on their Twitter activity | Uses large language models to identify opinions without needing specific training data. Works well across different political situations and elections. | Relying on pre-trained models might not work well for specific topics that weren't included in the training data. Since it doesn't use labeled training data, it might make mistakes in detecting opinions. |

## 2.2 Gaps Identified from The Literature Survey

The following are the research gaps identified from the literature review encompassing the work of ten different authors:

- Existing stance detection models often rely on domain-specific datasets, limiting their generalizability across different social media platforms and global contexts.
- Many deep learning-based stance detection models require high computational resources, making real-time analysis challenging in low-resource environments.
- Current methods struggle with detecting stance in multilingual or code-mixed text, which is common in social media discourse.
- Models using graph-based approaches face scalability issues when applied to large-scale, real-world datasets.
- Few-shot learning methods for stance detection show promising results but are highly dependent on the quality and diversity of the training data.

## **CHAPTER 3**

### **SYSTEM ANALYSIS**

## **CHAPTER 3**

### **SYSTEM ANALYSIS**

#### **3.1 System Analysis**

The Multi-Target Stance Detection System is an advanced natural language processing (NLP) model designed to classify stance on multiple targets within text data. Using deep learning methods, especially a Multi-Target Adversarial Attention Network (MTAAN), the system processes inputs like text content, context relationships, and sentiment changes to determine if a given statement is in favor of, against, or neutral towards specific targets. Built with a BERT-based encoder and adversarial training methods, the model improves its ability to clearly tell the difference between stance and sentiment, tackling key challenges in multi-target stance detection.

The system's main goal is to automate and improve stance classification, cutting down manual work while boosting accuracy and flexibility across different topics. By using attention methods to highlight important text parts, the model guarantees a data-driven, scalable, and effective way to detect stance. In the end, the Multi-Target Stance Detection System gives researchers, analysts, and businesses useful insights into public opinions, helping them make informed decisions in various areas.

#### **3.2 Problem definition**

The problem addressed by the Multi-Target Stance Detection with Adversarial Attention Network is the difficulty of accurately figuring out a speaker's stance on multiple targets in each text, especially when targets are not clearly stated or are new to the model. Traditional stance detection methods usually focus on single-target classification, which limits their ability to adapt across different areas and manage complex links between stance, sentiment, and topic.

Additionally, current models have trouble with understanding context and adapting, which causes misclassification and lower accuracy in real-world uses like social media analysis, political discussion evaluation, and sentiment tracking. To fix these issues, the proposed system uses a Multi-Target Adversarial Attention Network (MTAAN), including a BERT-based encoder and adversarial training methods to improve the model's ability to tell the difference between sentiment and stance. By improving

stance classification accuracy, adaptability, and robustness, the system aims to provide a more reliable and scalable solution for analyzing opinions and viewpoints in diverse and evolving contexts.

### **3.3 Existing System**

In the current stance detection models, deep learning techniques like Long Short-Term Memory (LSTM) networks and Convolutional Neural Networks (CNN) are often used to determine a text's stance toward a specific target. While these models can achieve decent accuracy, they often have trouble when it comes to detecting stances on multiple targets at once. They mainly rely on patterns they learn from data, which can limit their ability to understand the complex context that shapes a speaker's stance. Additionally, they may struggle with new targets because they lack the flexibility needed for changing real-world situations, such as analyzing social media or tracking public opinion.

Even though LSTM and CNN models have shown potential, they have important limitations in dealing with complex language and the connections between stance, sentiment, and context. LSTMs are good for processing sequences of data but can have difficulty remembering information from far back in the text and often need a lot of training data to work well. On the other hand, CNNs can identify local patterns in text but may not effectively capture the order of words, which can lead to mistakes, especially when multiple targets are mentioned in one text. These limitations make current systems less flexible and reliable for classifying stances on multiple targets. Therefore, there is a growing need for better models that can tackle these challenges by using contextual information and advanced learning techniques to improve stance detection accuracy.

#### **3.3.1 Disadvantages of Existing System**

Fixing the limits of current stance detection models is important for improving classification accuracy and ensuring reliable results in different areas. The development of advanced models, such as the Multi-Target Adversarial Attention Network (MTAAN), aims to overcome these challenges by employing deep learning and adversarial training techniques to enhance stance detection performance in complex text dataset.

***Limited Accuracy:***

Traditional stance detection models, like Long Short-Term Memory (LSTM) and Convolutional Neural Network (CNN), show limited accuracy when handling multi-target stance detection. While these models may perform well on structured and clear datasets, they struggle with real-world text that contains hidden, vague, or indirect stance expressions. Their inability to fully capture context and language nuances often leads to misclassifications, reducing reliability in practical applications like social media analysis and sentiment tracking.

***Inability to Capture Complex Relationships:***

Stance detection involves intricate connections between sentiment, topic context, and language structure, which traditional deep learning models may not fully capture. LSTMs, while effective for sequential data, can sometimes overlook important contextual information, while CNNs may not adequately model the relationships between words in a sequence. As a result, these models may struggle to distinguish stance-related elements from sentiment, leading to incorrect stance predictions in complex statements.

***Limited Scalability:***

The current stance detection methods may not be ideal for processing large volumes of text. When analyzing extensive datasets from social media, news, or online discussions, LSTM and CNN models can become computationally intensive, leading to slower performance and increased resource requirements.

***Potential Overfitting:***

Deep learning models like LSTMs and CNNs can overfit when trained on small or specific datasets. Overfitting causes these models to perform well on training data but struggle to adapt to new topics or targets, resulting in poor stance classification on fresh or evolving subjects. This diminishes the flexibility and effectiveness of the stance detection system, making it unreliable for dynamic real-world applications.

***Limited Adaptability:***

Current stance detection systems may lack the flexibility to adapt to new or unseen targets, which limits their effectiveness in areas where stances change over time. Pre-trained models without adversarial learning may struggle to generalize beyond their training data, leading to poor performance on trending topics, political debates, or new discussions. Their inability to adjust to changing language patterns renders these models unsuitable for real-world stance detection tasks.

***Lack of Transparency:***

Many deep learning models operate as black-box systems, making it difficult for users to understand how stance classification decisions are made. LSTMs and CNNs do not provide clear explanations for why a specific stance was assigned, reducing trust in automated stance detection. This lack of explainability complicates the use of these models in critical areas like policy analysis, legal reviews, and media monitoring, where transparency is essential for informed decision-making.

**3.4 Proposed System**

In the proposed stance detection model, a Multi-Target Adversarial Attention Network (MTAAN) is utilized to boost accuracy, robustness, and interpretability. By merging adversarial training techniques with attention mechanisms, the model significantly overcomes the limitations of existing systems, ensuring a more precise and scalable approach to stance detection.

At the heart of the MTAAN model is a BERT-based encoder that extracts contextual embeddings from text, capturing the complex relationships between stance, sentiment, and target context. Traditional stance detection models depend on fixed, predefined features that restrict their adaptability, but BERT allows for dynamic learning. This enables the system to grasp language variations, implicit references, and changing topics. As a result, the model can generalize better across different datasets, ensuring greater reliability in detecting stances on both familiar and new targets...

A key feature of the proposed system is its adversarial training mechanism, which helps the model distinguish between stance-related and sentiment-related information. Many current models mix up stance and sentiment, resulting in incorrect

classifications. However, MTAAN uses adversarial learning with target and sentiment discriminators that challenge the stance detection module during training. This process compels the model to sharpen its understanding of stance, improving accuracy and reducing bias when handling complex, unclear, or multi-target texts.

The attention mechanism further improves the model's performance by enabling it to focus on the most relevant parts of a text. Unlike traditional models that treat all words the same, the attention mechanism highlights important phrases, context clues, and language patterns that indicate stance. This allows the system to accurately classify stances even when targets are mentioned indirectly, addressing a major limitation of earlier methods. The ability to identify stance implicitly makes the model very effective in social media conversations, news articles, and online debates, where direct mentions of targets are not always available.

In conclusion, the proposed stance detection system marks a significant advancement over traditional methods by combining deep learning, adversarial training, and attention mechanisms. Its capability to accurately classify stance across multiple targets, adapt to new topics, and reduce overfitting risks makes it a highly reliable and scalable solution for stance detection in real-world applications.

#### **3.4.1 Advantages of Proposed System**

The proposed multi-target stance detection system offers a more advanced and efficient way to analyze stances in text, addressing the shortcomings of traditional methods. By combining deep learning, adversarial training, and attention mechanisms, the system greatly improves the accuracy, scalability, and adaptability of stance detection. This enhancement makes it a valuable tool for analyzing public opinions, social media discussions, political debates, and sentiment analysis across various fields.

##### ***Enhanced accuracy:***

The proposed system uses a multi-target adversarial attention network, which significantly boosts stance detection accuracy compared to traditional machine learning models like logistic regression and support vector machines. By utilizing a BERT-based encoder, the system effectively captures contextual relationships,

allowing it to classify stance with greater precision. The adversarial training technique further sharpens the model's ability to distinguish between sentiment and stance, reducing misclassifications and enhancing overall performance. This improved accuracy ensures that the system can analyze complex textual data more reliably, making it suitable for real-world applications such as opinion mining, policy analysis, and evaluating public discourse.

***Ability to capture complex relationships:***

A major strength of the proposed system is its ability to identify intricate relationships between stance, sentiment, and topic context. Unlike traditional models that depend on predefined features and statistical patterns, this model uses deep learning-based attention mechanisms to focus on the most relevant parts of the text. This approach enables the system to recognize implicit stance expressions, even when targets are not directly mentioned. Additionally, the multi-target classification capability allows the model to handle texts discussing multiple subjects, making it more effective for or large-scale stance detection tasks, especially in diverse and dynamic environments.

***Robust and adaptive framework:***

The adversarial training mechanism incorporated in the model significantly enhances its robustness and adaptability. Unlike traditional stance detection models that often struggle with unseen targets, the proposed system is designed to generalize across different domains and new stance topics. By continuously improving its ability to differentiate between stance and sentiment, the system adapts to evolving linguistic patterns, making it highly effective for analyzing dynamic and rapidly changing online discussions. This adaptability ensures that the model remains relevant even as new topics emerge in social media and news articles, allowing for timely and accurate analysis in various contexts.

***Interpretability and transparency:***

The proposed stance detection model is designed to be more interpretable than traditional black-box machine learning models. The use of attention mechanisms provides insights into which parts of the text contribute to stance classification, allowing researchers and analysts to understand the reasoning behind the model's predictions. This transparency fosters trust and confidence among users, making the

system suitable for applications where explainability is critical, such as journalism, legal analysis, and academic research. The ability to interpret stance classification decisions also enhances the reliability of automated sentiment and opinion mining tools, ensuring that users can make informed decisions based on the model's outputs.

***Improved scalability:***

The proposed stance detection model is designed to be more interpretable than traditional black-box machine learning models. The use of attention mechanisms provides insights into which parts of the text contribute to stance classification, allowing researchers and analysts to understand the reasoning behind the model's predictions. This transparency fosters trust and confidence among users, making the system suitable for applications where explainability is critical, such as journalism, legal analysis, and academic research. The ability to interpret stance classification decisions also enhances the reliability of automated sentiment and opinion mining tools, ensuring that users can make informed decisions based on the model's outputs.

***Reduced risk of overfitting:***

By incorporating adversarial training and advanced optimization techniques, the proposed system minimizes the risk of overfitting. Traditional stance detection models often suffer from overfitting when trained on limited datasets, leading to poor performance on unseen data. The proposed system addresses this issue by employing data augmentation techniques, adversarial learning strategies, and regularization methods, ensuring better generalization across different datasets. This results in more reliable stance classification, even when analyzing previously unseen topics or linguistic variations, ultimately enhancing the model's effectiveness in diverse real-world applications.

### **3.5 Feasibility Study**

The feasibility study aims to evaluate the viability of the proposed multi-target stance detection system by analyzing its technical, operational, and economic aspects. This assessment ensures that the system can be successfully developed, deployed, and maintained while providing significant benefits over existing models. By examining these key areas, we can determine whether the proposed stance detection model is

practical, efficient, and cost-effective for real-world applications such as social media analysis, political discourse tracking, and sentiment monitoring.

- Technical Feasibility
- Operational Feasibility
- Economical Feasibility

***Technical Feasibility:***

Technical feasibility assesses whether the proposed stance detection model can be developed and implemented using current technologies, tools, and computational resources. The system is built using deep learning frameworks such as TensorFlow and PyTorch, which are widely available and extensively used for natural language processing tasks. The integration of a BERT-based encoder ensures that the model can efficiently extract contextual embeddings from textual data, enabling accurate stance classification. Additionally, adversarial training mechanisms and attention-based models improve the robustness of stance detection, making it more adaptable to real-world scenarios.

***Operational Feasibility:***

Operational feasibility assesses the ease of implementation, user accessibility, and practical functionality of the proposed stance detection system. As the model is designed for multi-target stance classification, it must integrate seamlessly with existing media monitoring tools, business intelligence platforms, and sentiment analysis applications. The system provides API-based access and intuitive dashboards, enabling researchers, analysts, and decision-makers to retrieve and interpret stance classification results with ease and efficiency.

The attention-based mechanism embedded in the model enhances its interpretability, making it easier for users to understand why a particular stance classification was assigned. This transparency fosters trust and confidence in the system, ensuring that users can rely on its outputs for applications such as policy analysis, brand reputation management, and news sentiment tracking. By providing clear insights into the decision-making process, the model empowers users to make informed decisions

based on the stance detection results, ultimately improving the effectiveness of their analyses.

***Economic Feasibility:***

Economic feasibility examines whether the benefits of the proposed stance detection model justify its development, implementation, and maintenance costs. While the initial costs encompass software development, data collection, model training, and computational infrastructure, these investments are offset by long-term advantages, such as improved accuracy, automation, and reduced reliance on manual stance classification. This model allows for cost-effective expansion and adaptation to meet the needs of various user communities. By streamlining processes and enhancing efficiency, the system ultimately provides a strong return on investment, making it a financially viable solution for organizations seeking to leverage stance detection technology.

By leveraging pre-trained models and cloud-based deployment, the system reduces computational costs compared to traditional methods that require extensive custom training. Cloud-based solutions also eliminate the need for expensive local hardware, enabling cost-effective scalability as data volumes increase.

# **CHAPTER 4**

## **REQUIREMENTS SPECIFICATIONS**

## **CHAPTER 4**

### **REQUIREMENTS SPECIFICATIONS**

#### **4.1 Purpose, Scope, Definition**

##### **4.1.1 Purpose**

The purpose of the software requirements specification is to serve as the base for the entire project by defining the expectations, functions, and constraints of the system. It provides a clear guide that all teams involved in the development process must follow, ensuring consistency and alignment throughout the project. This document acts as a reference for developers, quality assurance teams, operations, and maintenance staff, helping them understand the scope and needs of the system to be built.

##### **4.1.2 Scope**

The scope of the software requirements specification plays an important role in project planning by determining and recording the key goals, deliverables, features, and functions that the system must support. It sets clear limits for the project, defining the work that needs to be done to deliver a fully working software solution. By outlining the tasks, deadlines, and estimated costs, the software scope ensures that the development process stays organized and manageable. A well-defined software scope helps in identifying the functions and technical parts that will be included in the final product. It specifies the actions needed to achieve the project goals, ensuring that all stakeholders have a shared understanding of what the system will do. By setting clear expectations, the software scope reduces misunderstandings, scope creep, and possible conflicts that could arise during the development phase.

##### **4.1.3 Definition**

The Software Requirement specification is a clear document that describes the software system to be made, outlining its features, limits, and intended use. It serves as a guide for the development team, helping them in designing and building the system according to the stated needs. This document is arranged to ensure that the software meets business needs while following technical rules and industry best practices.

Modeled after business requirements specification, the software requirements specification defines the system in terms of its expected behavior, performance, and interactions with users and other systems. It translates stakeholder expectations into a

technical format that developers and engineers can use as a reference throughout the project lifecycle. By providing a structured description of the system, it ensures that all teams involved in the development process have a common understanding of the project goals and functionalities.

## **4.2 Requirement Analysis**

The Requirement analysis is the process of gathering, reviewing, and recording the software needs necessary for the development of a system. This process, also called requirements engineering, plays an important role in making sure that the final product meets the expectations of users and stakeholders. It involves defining and keeping a complete and organized Software Requirements Specification (SRS) document, which serves as a guide throughout the development lifecycle. This includes -

- Feasibility Study
- Requirements Gathering
- Software Requirements Specification
- Software Requirements Validation

The basic requirements of our project are:

- Research Papers
- Camera

A feasibility study checks if the proposed system can be built with the available resources, requirements gathering involves collecting input from stakeholders, software requirements specification describes the functions and limits of the system, and software requirements validation ensures that the defined needs are correct, complete, and match business needs.

### **4.2.1 Functional Requirement Analysis**

Functional requirements include text preprocessing, stance classification, sentiment detection, and multi-target stance detection. The system must be able to process input text, extract important features, use deep learning models for classification, and produce accurate stance predictions. Additionally, the system should support real-time data analysis, work with external applications, and automate result generation.

#### 4.2.2 User Requirements Analysis

User Requirements Analysis is the process of determining user expectations for a new or modified product. These features must be Quantifiable, relevant, and detailed.

#### 4.2.3 Nonfunctional Requirement Analysis

Non-functional requirements describe the general characteristics of a system. They are also known as quality attributes. Some typical non-functional requirements are Performance, Response Time, Throughput, Utilization, and Scalability.

***Performance:***

The performance of a device is essentially estimated in terms of efficiency, effectiveness, and speed.

- Short response time for a given piece of work.
- High throughput (rate of processing work)
- Short data transmission time.

***Response Time:***

Response is the time a system or functional unit takes to react to a given input

### 4.3 System Requirements

By System requirements define the software and hardware components necessary for the development, deployment, and execution of the stance detection.

#### 4.3.1 Software Requirements

- ***Operating System:*** Windows 10 or 11 Ultimate, Linux, Mac.
- ***Visual Studio Code:*** Utilized as the Integrated Development Environment (IDE) for writing and editing code, facilitating collaborative development, and managing project files.
- ***Google Colab:*** Used for data preprocessing, model training, and testing deep learning algorithms in a cloud-based Jupyter notebook environment, leveraging Google's computational resources.
- ***Packages:*** NumPy, Pandas, Scikit-learn, TensorFlow, Matplotlib.
- ***Streamlit:*** Used for the front-end development and user interface, allowing seamless interaction with the backend for disease classification.

#### 4.3.2 Hardware Requirements

- **Processor i3/i5:** The system can be developed and run on a standard PC equipped with an Intel Core i3 or i5 processor, ensuring sufficient processing power for development tasks and system operation.
- **RAM - 4GB:** A minimum of 4GB RAM is recommended to support multitasking and handle computational tasks efficiently during the development and deployment phases.
- **Hard Disk Drive - Minimum 500 GB:** Adequate storage space is required to store project files, datasets, and software installations. A minimum of 500 GB hard disk drive ensures ample storage capacity for the system's requirements.

# **CHAPTER 5**

## **SYSTEM DESIGN**

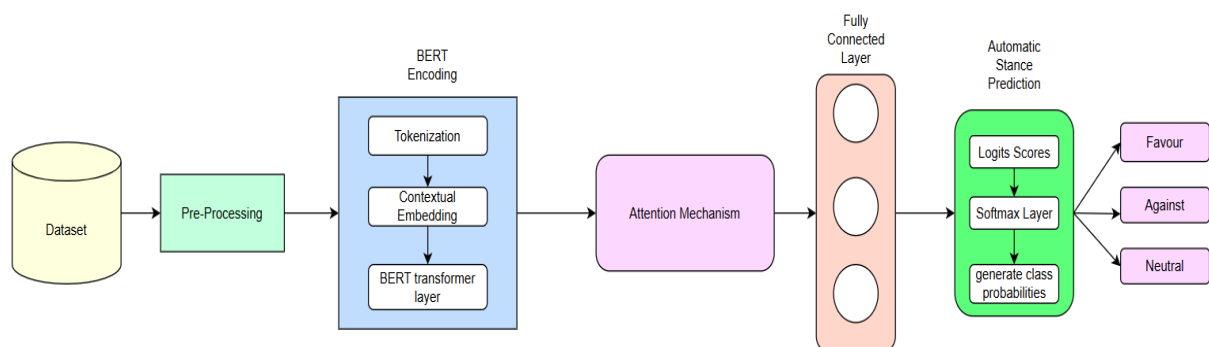
## CHAPTER 5

### SYSTEM DESIGN

#### 5.1 System Architecture

The project on stance detection focuses on identifying the stance expressed in text samples, such as "support," "against," or "neutral." It begins with selecting a suitable dataset containing labeled text data on various topics. Data preprocessing involves cleaning the text, normalizing it, tokenizing, and applying data augmentation techniques to enhance the dataset.

The preprocessed data is then split into training and testing sets to facilitate effective model training and evaluation. An appropriate algorithm, such as BERT (Bidirectional Encoder Representations from Transformers) with an Attention Mechanism, is chosen for classification. The model is trained on the training dataset and evaluated on the testing dataset using metrics like accuracy and F1 score. Finally, a web application is developed using Streamlit, allowing users to input text and receive stance predictions as output, providing a user-friendly interface for real-time analysis.



**Fig-5.1. System Architecture**

#### 5.2 Modules

There are 2 modules used. They are:

1. Bidirectional Encoder Representations from Transformers +Multi-Target Adversarial Attention Network Machine Learning Model
2. Streamlit Framework and Flask for User Interface

### 5.2.1 BERT with Multi-Target Adversarial Attention Network Deep Learning Model

Component diagrams illustrate the various modules and components of a software system, providing a visual representation of the model's structure and organization. In the Multi-Target Adversarial Attention Network (MTAAN) designed for stance detection, a BERT encoder, attention layer, pooling layer, fully connected layer, and softmax classifier are integrated. Furthermore, adversarial mechanisms involving target and sentiment discriminators are incorporated to enhance the model's generalization capabilities. This deep learning-based approach aims to accurately classify the stance (Favor, Against, Neutral) expressed in a tweet towards a given target by focusing on the most relevant linguistic features and mitigating biases related to specific targets or sentiment expressions.

#### MTAAN Architecture Details

MTAAN employs a modular architecture where each component contributes to the overall stance detection process:

**BERT Encoder (Embedding Layer):** The input, consisting of a tweet and its corresponding target, undergoes tokenization and is processed by a pre-trained BERT model. This layer serves as the initial feature extraction stage.

**BERT Output:** The BERT model generates deep contextual representations for each token within the input sequence.

**Semantic Encoding:** These embeddings effectively capture the semantic relationships existing between the tweet's content and the specified target.

**Output Dimension:** The resulting output tensor has a shape of  $[L \times D]$ , where  $L$  denotes the sequence length (number of tokens) and  $D$  represents the hidden size of the BERT-base model, which is 768.

**Attention Layer:** Following the BERT encoder, a linear attention layer is applied to the BERT output. This mechanism aims to identify and emphasize words that are particularly relevant to determining the stance expressed in the tweet.

**Attention Weight Learning:** The attention layer learns specific weights for each token embedding produced by BERT.

**Interpretability Enhancement:** By assigning higher weights to crucial tokens and lower weights to less informative ones, this layer contributes to the model's interpretability.

**Attention Calculation:** The attention weights ( $A$ ) are calculated using the following formula:  $A = \tanh(W_{\text{att}} \times H)$  where  $H$  represents the output from the BERT encoder, and  $W_{\text{att}}$  is a trainable weight matrix specific to the attention layer.

**Weighted Feature Map:** The BERT output is then weighted by the attention scores through element-wise multiplication (represented by the symbol  $\odot$  for element-wise multiplication) to produce a weighted feature map ( $H_{\text{weighted}}$ ):  $H_{\text{weighted}} = A \odot H$

**Pooling Layer:** To transform the sequence of attended token embeddings into a fixed-length vector representation, a pooling operation is applied.

**Mean Pooling:** Mean pooling is performed across the sequence dimension of the weighted feature map.

**Stance Information Aggregation:** The resulting pooled vector ( $v$ ) effectively summarizes the overall stance-related information present in the input tweet with respect to the target.  $v = \text{mean}(H_{\text{weighted}})$

**Fully Connected Layer:** The fixed-length pooled feature vector is then passed through a dense (fully connected) layer.

**Mapping to Class Logits:** This layer learns to map the aggregated representation to a set of class logits ( $z$ ), where the number of logits corresponds to the number of stance classes.  $z = W_{\text{cls}} \times v + b$  Here,  $W_{\text{cls}}$  and  $b$  represent the weights and biases of the classifier layer, respectively.

**Softmax Layer:** The final layer of the stance classification branch applies the softmax activation function to the class logits.

**Probability Generation:** The softmax function converts the logits into a probability distribution ( $y$ ) over the three stance classes: Favor, Against, and Neutral.  $y = \text{softmax}(z)$

**Stance Prediction:** The class with the highest predicted probability is considered the model's final stance prediction.

### **Incorporating Adversarial Mechanism**

To enhance the model's ability to generalize effectively across unseen targets and variations in sentiment, MTAAN incorporates two adversarial discriminator modules that operate in conjunction with the main stance classification branch:

**Target Discriminator:** This module is designed to predict the topic or target of the input tweet.

**Target Invariance Encouragement:** By training the target discriminator to accurately identify the target, the main stance model is incentivized to learn features that are invariant to the specific target being discussed, focusing instead on the pure expression of stance.

**Sentiment Discriminator:** This module aims to classify the sentiment (positive, negative, neutral) expressed in the tweet.

**Sentiment Overfitting Prevention:** By training the sentiment discriminator, the model is discouraged from relying solely on sentiment cues to determine stance, as sentiment towards a target does not always directly correlate with the stance taken on the target itself.

These discriminator modules are trained to challenge the stance classifier during the training process, thereby promoting the learning of more robust and generalizable stance features.

### **Joint Loss and Optimization**

The MTAAN model is trained using a joint loss function that combines the loss from the stance prediction task with the adversarial feedback received from the target and sentiment discriminators:

$$L_{\text{total}} = L_{\text{stance}} - \lambda_1 \times L_{\text{target}} - \lambda_2 \times L_{\text{sentiment}}$$

Where:

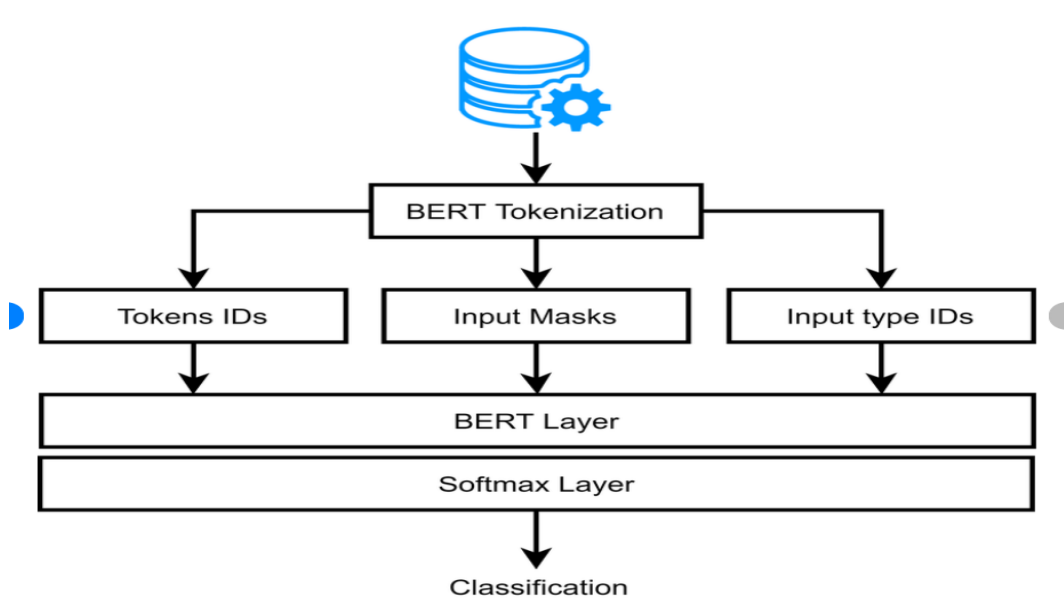
$L_{\text{stance}}$  represents the cross-entropy loss calculated for the stance classification task.

$L_{\text{target}}$  denotes the binary cross-entropy loss computed for the target discriminator.

$L_{\text{sentiment}}$  signifies the binary cross-entropy loss calculated for the sentiment discriminator.

$\lambda_1$  and  $\lambda_2$  are loss balancing factors that control the influence of the adversarial loss terms on the overall training objective.

**Optimization:** The entire MTAAN model is trained using the Adam optimization algorithm. This efficient gradient-based optimization method iteratively updates the model's weights to minimize the defined joint loss function, leading to improvements in both stance classification accuracy and generalization across diverse targets and sentiments.



**Fig 5.2 BERT Process**

### 5.2.2 Streamlit Framework with Flask for User Interface

Streamlit is an open-source app framework specifically designed for creating interactive web applications for machine learning and data science projects. When combined with Flask, you can quickly build user interfaces that allow users to interact with your models and visualize data while leveraging Flask's backend capabilities. Here's how Streamlit can be utilized for the user interface of your application:

Define the layout in Streamlit to create a clean and intuitive user interface. You can use various Streamlit components to display text, images, and interactive widgets, enabling users to input parameters and view results seamlessly. Streamlit's simple API allows you to create forms for user inputs, display model predictions, and provide visualizations of data without extensive coding. Flask can handle the backend logic, processing requests and returning results to the Streamlit frontend.

While Streamlit does not have a built-in ORM like Django, you can easily integrate it with data processing libraries such as Pandas and NumPy to handle data manipulation. This allows you to represent the structure of your data and perform necessary calculations or transformations before displaying results to the user.

Streamlit allows you to serve static files, such as images and CSS, to enhance the user interface. You can use these static files to style your application and improve the overall user experience. Define the main script in Streamlit to control the flow of your application. This script will manage user interactions, update the display based on inputs, and call the necessary Flask API endpoints to generate predictions or visualizations.

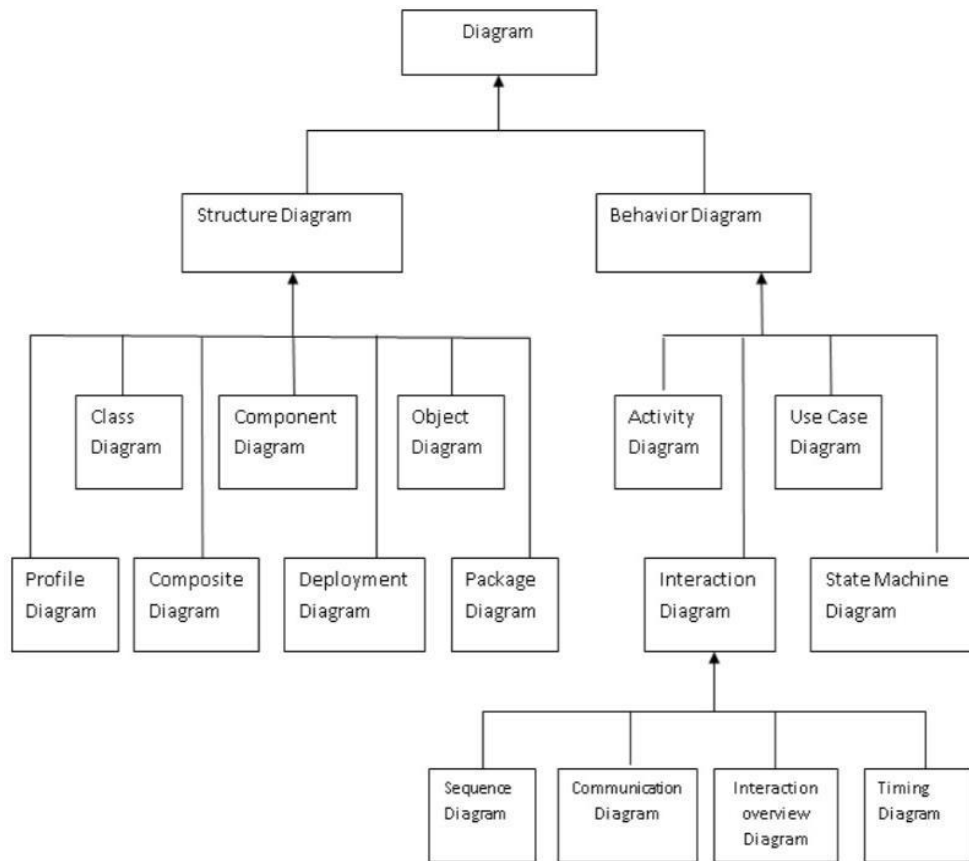
If needed, Streamlit supports session state management, which allows you to maintain user-specific data across interactions. This feature can be useful for personalizing user experiences or retaining input values during the session. By leveraging these features of Streamlit alongside Flask, you can create a user-friendly and interactive interface for your application, allowing users to input parameters and receive immediate feedback from the backend.

Streamlit comes with a set of built-in components that simplify the development of web applications, such as charts, tables, and media elements. This straightforward approach enables developers to focus on building their application logic without worrying about the complexities of web development. Streamlit follows a script-based architecture, where the entire application is defined in a single Python script. This makes it easy to understand and modify the application flow while integrating with Flask for backend processing.

Overall, Streamlit is a powerful and user-friendly framework that is well-suited for building interactive web applications for machine learning and data science projects. Its emphasis on simplicity, speed, and interactivity, combined with Flask's robust backend capabilities, has made it a popular choice among data scientists and developers looking to create engaging user interfaces.

### **5.3 Design Overview**

UML combines the best techniques from data modeling (entity relationship diagrams), business modeling (workflows), object modeling, and component modeling. It can be used with all processes, throughout the software development life cycle, and across different implementation technologies. UML has synthesized the notations of the Booch method, the Object modeling technique (OMT), and object-oriented software engineering (OOSE) by fusing them into a single, common and widely usable modeling language. UML aims to be a standard modeling language that can model concurrent and distributed systems.



**Fig-5.3. Types and categories of UML diagrams**

## 5.4 Uml Diagrams

The Unified Modeling Language (UML) is used to specify, visualize, modify, construct, and document the artifacts of an object-oriented software-intensive system under development. UML offers a standard way to visualize a system's architectural blueprints, including elements such as:

- Actors
- Business process
- (Logical) Components
- Activities
- Programming language statements
- Database schemas, and
- Reusable software components

The unified modeling language allows the software engineer to express an analysis model using the modeling notation that is governed by a set of syntactic-semantic and pragmatic rules. A UML system is represented using 5 different views that describe the system from distinctly different perspectives. Each view is defined by a set of diagrams, which are as follows:

***User Model View:***

- This view represents the system from the user's perspective.
- The analysis representation describes a usage scenario from the end-user's perspective.
- The UML user model view encompasses the models that define a solution to a problem as understood by the client stakeholders.

***Structural Model View:***

- In this model the functionality is arrived from inside the system.
- This model view models the static structures.

***Behavioral Model View:***

- It represents the dynamic of behavior as parts of the system, depicting the interactions of collection between various structural elements described in the user model and structural model view.

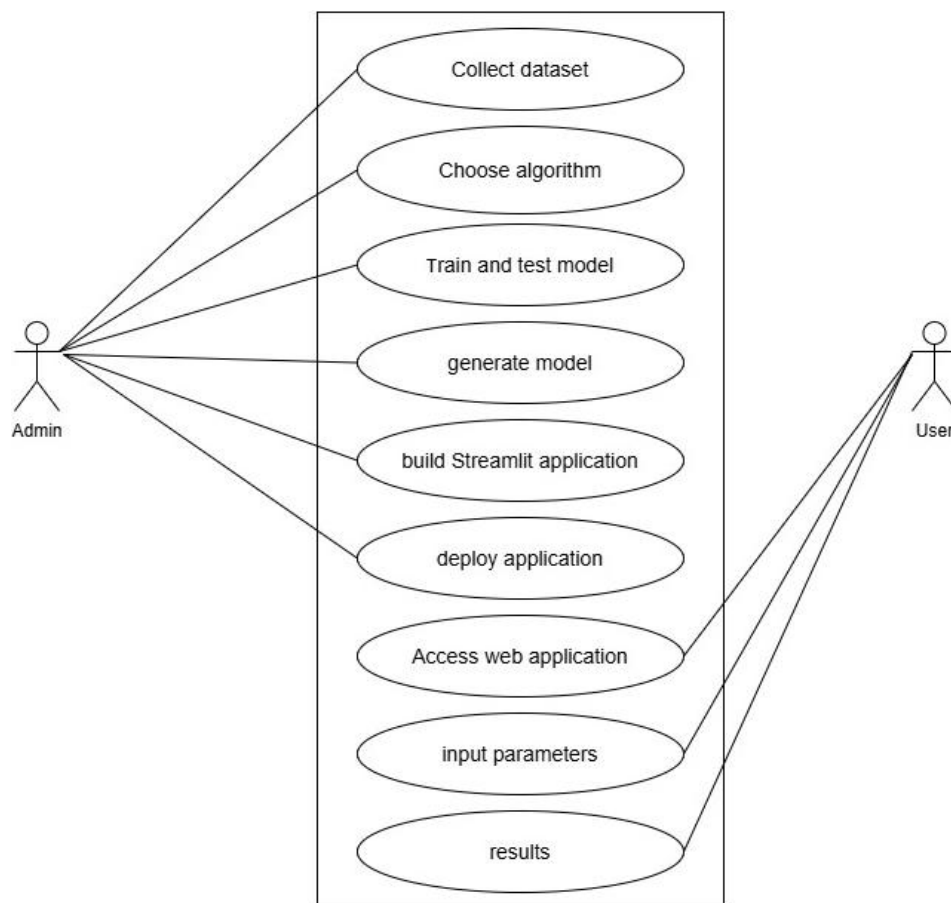
***Implementation Model View:***

- The implementation view is also known as the Architectural view which typically captures the enumeration of all the subsystems in the implementation model, the component diagrams illustrating how subsystems are organized in layers and hierarchies, and illustrations of import dependencies between subsystems.

***Environmental Model View:***

- These UML models describe both structural and behavioral dimensions of the domain or environment in which the solution is implemented. This view is often also referred to as the deployment or physical view.

### 5.4.1 Use case Diagram

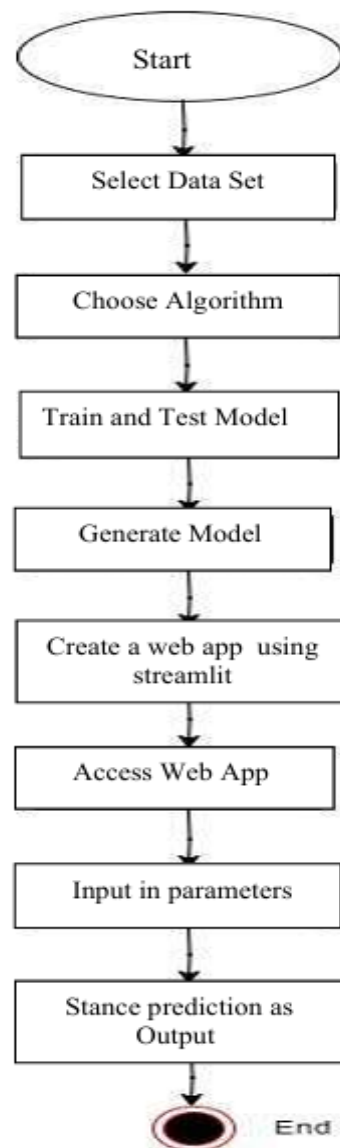


**Fig-5.4.1. Use Case Diagram**

A flow of events is a sequence of transactions performed by the system. They typically contain very detailed information, written in terms of what the system should do not how the system accomplishes the task flow of events are created as separate files or documents in your favorite text editor and then attached or linked to the use case using the files or documents in your favorite text editor and then attached or linked to a use case using the files tab of a model element.

Use case diagrams are usually referred to as behavior diagrams used to describe a set of actions (use cases) that some systems should or can perform in collaboration with one or more external users of the system (actors).

### 5.4.2 Activity Diagram



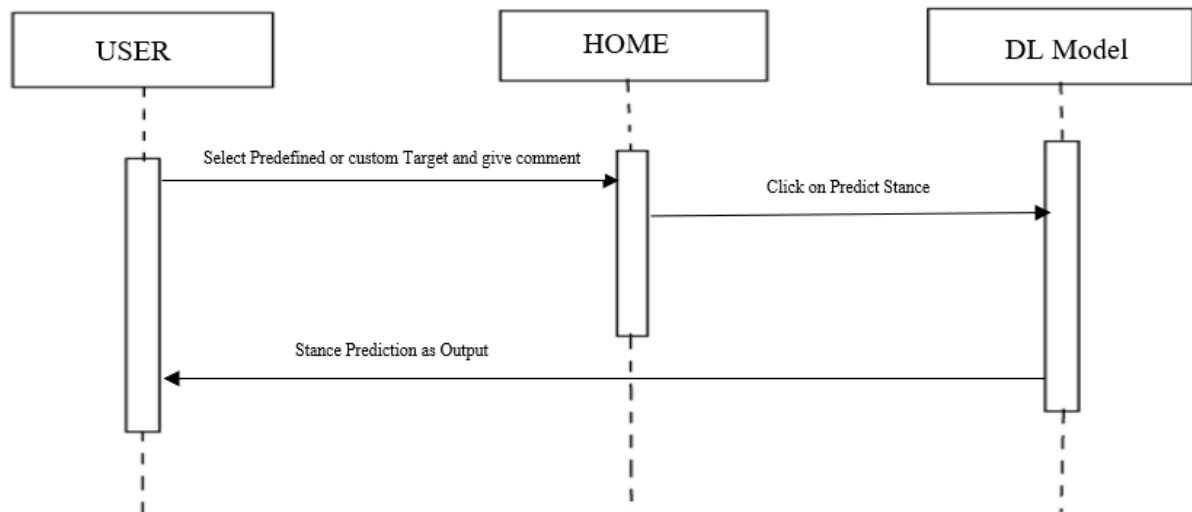
**Fig-5.4.2. Activity Diagram**

Activity Diagrams are graphical representations of Workflow of stepwise activities and actions with support for choice, iteration, and concurrency. In the Unified Modelling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

### 5.4.3 Sequence Diagram

A sequence diagram in UML is a kind of interaction diagram that shows how the process operates with one another and in what order. It is a construct of the Message

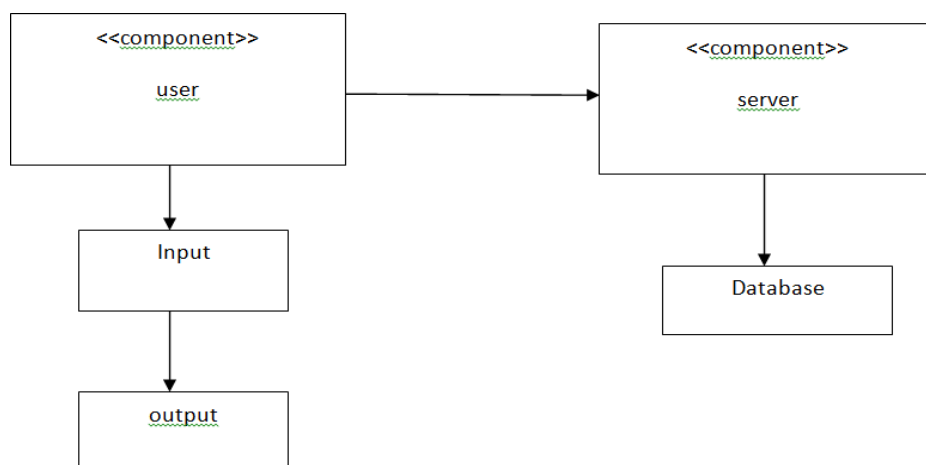
Sequence Chart. A sequence diagram shows, as parallel vertical lines (—lifelines), different processes or objects that live simultaneously, and, as horizontal arrows, the messages exchanged between them, in the order in which they occur. This allows the specification of simple runtime scenarios in a graphical manner.



**Fig-5.4.3. Sequence Diagram**

#### 5.4.4 Component Diagram

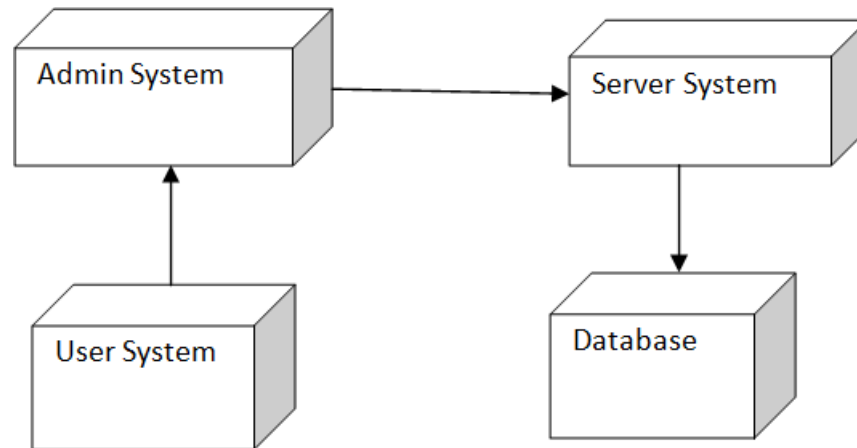
Component diagrams are used to display various components of a software system as well as subsystems of a single system. They are used to represent physical things or components of a system. It generally visualizes the structure and organization of a system.



**Fig-5.4.4 Component Diagram**

#### 5.4.5 Deployment Diagram:

A deployment diagram is a type of diagram that specifies the physical hardware on which the software system will execute. It also determines how the software is deployed on the underlying hardware.



**Fig-5.4.5. Deployment Diagram**

#### 5.5 Algorithm:

##### Algorithm for Proposed Model MTAAN:

**Input:** Tweet text and Target

**Output:** Predicted stance class (Favor, Against, Neutral)

**Step 1:** Initialize model parameters including BERT encoder weights, attention layer weights  $W_{att}$ , classifier weights  $W_{cls}$ , and biases. Also initialize discriminator modules for target and sentiment learning.

**Step 2:** Set hyperparameters:

Learning rate  $\eta$

Number of epochs  $N$

Batch size  $B$

Maximum sequence length  $L$

Loss weights for adversarial modules  $\lambda_1$  and  $\lambda_2$

**Step 3:** Preprocess the input data:

Tokenize the tweet and target using the BERT tokenizer.

Convert them into input IDs and attention masks.

Create dataset objects for training and testing.

**Step 4:** Define the model architecture:

BERT Encoder: Extract contextual embeddings from input sequences.

Attention Layer: Apply a trainable linear attention mechanism to focus on important words.

Feature Aggregation: Compute the weighted representation using:

$$A = \tanh(W_{\text{att}} \times H)$$

$$H_{\text{weighted}} = A \odot H$$

(Here,  $\odot$  denotes element-wise multiplication.)

Aggregate the representations using mean pooling:

$$v = \text{mean}(H_{\text{weighted}})$$

**Step 5:** Classification:

Pass the pooled representation through a fully connected layer to obtain class logits:

$$z = W_{\text{cls}} \times v + b_{\text{cls}}$$

Apply Softmax to get final stance probabilities:

$$y = \text{softmax}(z)$$

**Step 6:** Integrate adversarial discriminators:

Target Discriminator: Learns to distinguish target features.

Sentiment Discriminator: Learns to distinguish sentiment features.

These modules challenge the stance detector to learn generalized, task-robust features.

**Step 7:** Train the model:

Use CrossEntropyLoss for stance classification.

If adversarial learning is enabled, use a joint loss function:

$$L_{\text{total}} = L_{\text{stance}} - \lambda_1 \times L_{\text{target}} - \lambda_2 \times L_{\text{sentiment}}$$

Optimize using Adam optimizer and update weights via backpropagation.

**Step 8:** Evaluate the model using standard metrics:

Accuracy, Precision, Recall, F1-Score

Additional metrics:

$$\text{- RMSE} = \sqrt{\text{mean}((y_{\text{true}} - y_{\text{pred}})^2)}$$

$$\text{- MAE} = \text{mean}(|y_{\text{true}} - y_{\text{pred}}|)$$

$$\text{- MAPE} = \text{mean}(|(y_{\text{true}} - y_{\text{pred}}) / y_{\text{true}}|) \times 100$$

$$\text{- AUC} = \text{Area under ROC Curve (for multi-class: One-vs-Rest)}$$

**Step 9:** Return the predicted stance class with the highest probability (Favor, Against, or Neutral).

Predicted class =  $\text{argmax}(y)$

**Step 10:** Establish a Flask backend to handle prediction requests..

**Step 11:** Develop a web application using Streamlit to provide a user-friendly interface for stance detection, create input fields for users to enter text and select target topics.

Implement a function to send user input to the backend for prediction..

**Step 12:** Users can input fields for users to enter text and select target topics.

**Step 13:** The web application processes the user input using the trained MTAAN model and receive stance predictions in real-time.

**Step 14:** You will get the predicted stance to the user, providing feedback based on the input text and selected target.

The proposed algorithm leverages a BERT-based attention mechanism to classify tweet stances (Favor, Against, Neutral) in relation to a given target. Initially, the model tokenizes both the tweet and target text, processing them through a pre-trained BERT encoder to generate contextual embeddings. These embeddings capture semantic relationships and sentiment cues between the tweet and target.

To highlight the most relevant features, an attention layer is applied to focus on critical parts of the input, followed by mean pooling to aggregate the important features for stance prediction. The final classification is achieved using a fully connected layer with softmax activation to output one of the three stance categories: Favor, Against, or Neutral.

Adversarial discriminators are incorporated into the model to encourage robust generalization across different targets and sentiment nuances. The training process uses a joint loss function that combines stance prediction, target relevance, and sentiment classification losses. This ensures that the model simultaneously learns to classify stances accurately while maintaining adaptability to varying input characteristics.

The model is optimized using the Adam optimizer and evaluated on metrics such as accuracy, precision, recall, and F1-score. Once trained, it is deployed via a Flask backend and a Streamlit web application, enabling real-time stance predictions based on user input, making it accessible and interactive.

# **CHAPTER 6**

## **IMPLEMENTATION**

## CHAPTER 6

### IMPLEMENTATION

#### 6.1 Steps For Implementation

This section covers the Dataset, Preprocessing, and Implementation steps used in the proposed model for multi-stance detection. The dataset consists of text samples labeled by different stances. Preprocessing includes cleaning the text, normalizing it, and augmenting the data to improve model performance. Finally, the implementation is done using Python, utilizing deep learning libraries for training and evaluating the model.

##### 6.1.1 Dataset

The stance detection dataset used in this study is derived from the SemEval-2016 Task 6 competition. This dataset consists of 5 different targets: Feminist Movement, Hillary Clinton, Legalization of Abortion, Atheism, and Climate Change is a Real Concern. The stance for each target is categorized into three possible labels: Favor, Against, and Neutral.

The dataset contains a total of 5,628 instances, which are evenly split between training and test sets, with 2,814 instances in each split. Each instance represents a tweet expressing an opinion on one of the given targets. The dataset is structured to ensure balanced representation across all targets, enabling effective training and evaluation of stance detection models.

| Target                           | Training set size | Test set size | Total       |
|----------------------------------|-------------------|---------------|-------------|
| Feminist Movement                | 664               | 664           | 1328        |
| Hillary Clinton                  | 639               | 639           | 1278        |
| Legalization of Abortion         | 603               | 603           | 1206        |
| Atheism                          | 513               | 513           | 1026        |
| Climate Change is a Real Concern | 395               | 395           | 790         |
| <b>Total Instances</b>           | <b>2814</b>       | <b>2814</b>   | <b>5628</b> |

**Table 6.1.1 Dataset**

### 6.1.2 Data Preprocessing

Before training the MTAAN (Multi-Task Attention Augmented Network) model, a crucial step was the preprocessing of the dataset to ensure that the input text and corresponding targets were appropriately formatted for a BERT-based architecture. The dataset was loaded from Excel files using the panda's library, containing columns for the main text, associated target, and their stance labels.

To convert raw textual data into numerical representations compatible with the BERT model, the Bert Tokenizer from the Hugging Face Transformers library was used. Each text-target pair was tokenized with padding and truncation applied to maintain a uniform sequence length of 128 tokens, ensuring consistency across batches. The processed inputs, including input IDs and attention masks, were then prepared for efficient batching and training using PyTorch's custom Dataset and Data Loader utilities.

In the Stance Dataset class, each data sample was individually tokenized, and labels were converted into PyTorch tensors to facilitate supervised learning. This preprocessing step not only enabled seamless integration with the BERT backbone but also optimized memory usage during training. By squeezing unnecessary tensor dimensions and leveraging attention masks, the model was equipped to better focus on relevant parts of the input sequence.

The careful structuring of the dataset and attention to detail during preprocessing contributed significantly to the model's ability to learn contextual representations, ultimately enhancing its stance classification performance across favor, against, and neutral labels. This preprocessing pipeline laid the groundwork for the successful deployment of the BERT + MTAAN architecture in a stance detection task.

### 6.1.3 Implementation on Python

What is a Script?

A script is a type of computer program written in a scripting language that contains a sequence of commands in a file. Unlike traditional programming languages, scripts can be run directly without needing to be compiled first. This feature is helpful because it

allows us to write a program and see the results right away in an interactive way.

- Scripts can be reused in different projects.
- Scripts can be easily modified and updated

### ***Difference between a script and a program***

#### ***Script:***

Scripts are distinct from the core code of the application, which is usually written in a different language, and are often created or at least modified by the end-user. Scripts are often interpreted from source code or byte code, whereas the applications they control are traditionally compiled to naïve machine code.

#### ***Program:***

The program has an executable from that the computer can use directly to execute the instructions. The same program in its human-readable source code form, from which executable programs are derived (e.g., compiled).

#### ***Python:***

What is Python? Python is an interpreter, high-level, general-purpose programming language. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming.

#### ***Python concepts:***

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- Python is interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive - You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- Python is Object-Oriented – Python supports Object-Oriented style or technique of
- programming that encapsulates code within objects.
- Python is a Beginner's Language – Python is a great language for the beginner-

level

- programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

### ***History of Python***

Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.

Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, Smalltalk, and Unix shell and Other scripting languages.

Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).

Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

### ***Python Features***

python's features include

- Easy-to-learn: Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- Easy-to-read: Python code is more clearly defined and visible to the eyes.
- Easy-to-maintain: Python's source code is fairly easy-to-maintain.
- A broad standard library: Python's bulk of the library is very portable and cross-platform compatible on UNIX, windows, and Macintosh.
- Interactive Mode: Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- Portable: Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- Extensible: You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- Databases: Python provides interfaces to all major commercial databases.
- GUI Programming: Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.

- Scalable: Python provides a better structure and support for large programs

***Python modules:***

Python allows us to store our code in files(also called modules). To support this, Python has a way to put definitions in a file and use them in a script or in an interactive instance of the interpreter. Such a file is called a module; definitions from a module can be imported into other module or into the main module.

***Testing code:***

- Code is usually developed in a file using an editor.
- To test the code, import it into a python session and try to run it.
- Usually there is an error, so you can check by go to file, make a correction, and test again. This process is repeated until you are satisfied that the code works. The entire process is known as the development cycle.

***Implementation of Multi-Stance for Adversarial Attention Network***

The implementing the Multi-Target Adversarial Attention Network (MTAAN) can vary based on the specific task, data, and methods used. However, here's a general outline of the steps involved in a typical project:

***Define the Problem:*** Clearly understand and define the problem you want to solve. What stance are you trying to detect in social media posts? What opinions or sentiments are you aiming to classify?

***Gather Data:*** Gather relevant social media data that includes various stances on different topics. Ensure the dataset is diverse and representative.

***Data Preprocessing:***

***Data Cleaning:*** Clean and prepare the data for analysis. This may include tokenization, removing noise and handling missing values

***Choose a Model:*** Select the MTAAN architecture, which includes a pre-trained BERT encoder and adversarial attention mechanisms.

**Train the Model:** Use the training data to teach the model how to identify stances, optimizing the stance detection module while training the sentiment and target discriminators.

**Validate the Model:** Use the validation set to evaluate the performance of the trained model. Adjust hyperparameters or try different algorithms as necessary to improve performance.

**Evaluate the Model:** Test the model's performance on a separate dataset to see how accurately it detects stances and sentiments.

**Hyperparameter Tuning:** Adjust the model settings and hyperparameters to improve accuracy and performance based on evaluation results.

**Deploy the Model:** Implement the trained model in a real-world application to analyze opinions and sentiments in online discussions.

**Monitor and Maintain:** Keep an eye on how well the Multi-Target Adversarial Attention Network (MTAAN) performs once it is in use. Regularly check its accuracy and make updates as needed to adapt to new data trends or changes that could impact its performance.

**Documentation:** Write down everything about the project, including where the data came from, how it was cleaned and prepared, the model chosen, the training process, and the evaluation results.

### ***Implementation of Streamlit and Flask using Deep Learning***

Implementing Streamlit and Flask with deep learning involves integrating deep learning models into web applications using these frameworks.

#### ***Setup Streamlit and Flask Project:***

Install Streamlit and Flask if you haven't already: `pip install streamlit flask`. Create a new directory for your project and navigate into it `mkdir myproject cd myproject`. Create a new Flask application Create a file named `app.py` and set up a basic Flask application.

***Prepare Deep Learning Model:***

Train and save your deep learning model using libraries like TensorFlow or PyTorch. For instance, you can use a pre-trained BERT model for stance detection.

***Define Flask Views:***

Create views in your Flask app that will handle incoming requests and responses.

Design HTML templates to render the web pages. These templates will contain forms for user input and display the results returned by the views.

***Define URLs:***

Configure URL patterns in the `urls.py` file of your Django app to map URLs to the corresponding views. Specify the URLs where users can access the machine learning functionality within your web application.

***Handle Form Submission:***

Write logic in your views to handle form submissions.. Extract input data from the form, preprocess it if necessary, and pass it to the machine learning model for prediction. Return the predictions to the user interface for display.

***Integrate Machine Learning Model:***

Load the serialized machine learning model in your Django views. Use the model to make predictions based on the input data received from the user.

***Test:***

Test your Django web application locally to ensure that everything is working as expected. Submit sample data through the forms and verify that the predictions returned by the machine learning model are accurate.

***Deploy:***

Once you're satisfied with the functionality of your Django web application, deploy it to a web server or platform such as Heroku, AWS, or PythonAnywhere. Make sure to include any necessary dependencies, such as the machine learning libraries, in your deployment environment.

***Monitor and Maintain:***

Monitor your deployed application for any errors or performance issues. Update the machine learning model as needed with new data or retraining to improve accuracy over time.

**6.2 Coding****Deep Learning Code****1. *Importing necessary modules***

```
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Dataset
from transformers import BertTokenizer, BertModel
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, mean_absolute_error, mean_squared_error, roc_auc_score
import numpy as np
```

**2. *Constants***

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
EPOCHS = 50
BATCH_SIZE = 16
MAX_LEN = 128
```

**3. *Loading Dataset***

```
train_df = pd.read_excel("/mnt/data/augmented_trainingdata.xlsx")
test_df = pd.read_excel("/mnt/data/modified_testdata_taskA.xlsx")
```

**4. *Initializing Tokenizer***

```
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

**5. *Dataset Class Definition***

```
class StanceDataset(Dataset):
    def __init__(self, dataframe, tokenizer, max_len):
        self.data = dataframe
        self.tokenizer = tokenizer
        self.max_len = max_len
```

```
def __len__(self):
    return len(self.data)

def __getitem__(self, index):
    row = self.data.iloc[index]
    text, target, label = row['text'], row['target'], row['label']

    encoding = self.tokenizer(
        text, target, padding='max_length', truncation=True,
        max_length=self.max_len, return_tensors="pt"
    )

    return {
        'input_ids': encoding['input_ids'].squeeze(0),
        'attention_mask': encoding['attention_mask'].squeeze(0),
        'label': torch.tensor(label, dtype=torch.long)
    }
```

#### 6. *Create Dataset and DataLoader*

```
train_dataset = StanceDataset(train_df, tokenizer, MAX_LEN)
test_dataset = StanceDataset(test_df, tokenizer, MAX_LEN)
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE,
                           shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE,
                          shuffle=False)
```

#### 7. *Model Definition (MTAAN + BERT)*

```
class BERT_MTAAN(nn.Module):
    def __init__(self):
        super(BERT_MTAAN, self).__init__()
        self.bert = BertModel.from_pretrained("bert-base-uncased")
        self.attention = nn.Linear(768, 768)

        self.classifier = nn.Linear(768, 3) # 3 classes: Favor, Against, Neutral
        self.softmax = nn.Softmax(dim=1)

    def forward(self, input_ids, attention_mask):
        bert_output = self.bert(input_ids, attention_mask=attention_mask)
```

```
        .last_hidden_state d
    attn_weights = torch.tanh(self.attention(bert_output))
    weighted_rep = attn_weights bert_output
    pooled_output = torch.mean(weighted_rep, dim=1)
    logits = self.classifier(pooled_output)
    return logits
```

#### **8. Initialize Model, Loss Function, and Optimizer**

```
model = BERT_MTAAN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=2e-5)
```

#### **9. Training Loop**

```
for epoch in range(EPOCHS):
    model.train()
    total_loss = 0
    for batch in train_loader:
        input_ids, attention_mask, labels = batch['input_ids'].to(device), batch
            ['attention_mask'].to(device), batch['label'].to(device)
        optimizer.zero_grad()
        outputs = model(input_ids, attention_mask)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1}/{EPOCHS}, Loss: {total_loss/len(train_loader):.4f}")
```

#### **10. Evaluation**

```
model.eval()
y_true, y_pred, y_probs = [], [], []
with torch.no_grad():
    for batch in test_loader:
        input_ids, attention_mask, labels = batch['input_ids'].to(device),
            batch['attention_mask'].to(device), batch['label'].to(device)
        outputs = model(input_ids, attention_mask)

        probs = torch.softmax(outputs, dim=1)
        preds = torch.argmax(probs, dim=1)
        y_true.extend(labels.cpu().numpy())
        y_pred.extend(preds.cpu().numpy())
```

```
y_probs.extend(probs.cpu().numpy()[0, 1]) # Prob for class 1
```

```
auc = roc_auc_score(y_true, y_probs, multi_class='ovr') 100
```

### 11. Metrics Calculation

```
accuracy = accuracy_score(y_true, y_pred) 100
```

```
precision = precision_score(y_true, y_pred, average='weighted') 100
```

```
recall = recall_score(y_true, y_pred, average='weighted') 100
```

```
f1 = f1_score(y_true, y_pred, average='weighted') 100
```

```
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
```

```
mae = mean_absolute_error(y_true, y_pred)
```

```
mape = np.mean(np.abs((np.array(y_true) - np.array(y_pred)) / np.array(y_true)))  
100
```

```
auc = roc_auc_score(y_true, y_probs, multi_class='ovr') 100
```

### 12. Print Metrics

```
print("BERT + MTAAN Test Metrics:")
```

```
print(f"Accuracy: {accuracy:.2f}%")
```

```
print(f"Precision: {precision:.2f}%")
```

```
print(f"Recall: {recall:.2f}%")
```

```
print(f"F1-Score: {f1:.2f}%")
```

```
print(f"RMSE: {rmse:.4f}")
```

```
print(f"MAE: {mae:.4f}")
```

```
print(f"MAPE: {mape:.2f}%")
```

```
print(f"AUC: {auc:.2f}%")
```

## Streamlit Code

### *frontend.py code*

```
import streamlit as st
```

```
import requests
```

```
import base64
```

```
# Set Page Config
```

```
st.set_page_config(page_title="Stance Detection - MTAAN", page_icon="🧠",  
layout="centered")
```

```
# Function to set background image
```

```
def set_background(image_path):

    with open(image_path, "rb") as img_file:

        base64_str = base64.b64encode(img_file.read()).decode()

    bg_image = f"""

<style>

.stApp {{

    background-image: url("data:image/jpg;base64,{base64_str}");

    background-size: cover;

    background-position: center;

    background-attachment: fixed;

}}

</style>

"""

    st.markdown(bg_image, unsafe_allow_html=True)

# Set Background Image

set_background("C:/Users/ganta.greshma/stance-detection-app/image.jpg") # Adjust
path as needed

# Project Title

st.markdown(

    "<h1 style='text-align: center; color: red;'>Stance Detection - Multi-Target
Adversarial Attention Network</h1>",

    unsafe_allow_html=True,

)

# Define Target Topics

targets = [

    "Climate Change", "Gun Control", "Feminism", "Abortion", "Healthcare",

    "Immigration", "Animal Rights", "LGBTQ+ Rights", "Cryptocurrency",
```

```
"AI in Society", "Space Exploration", "Other"

]

# User Input Fields

st.markdown("### Enter Your Text and Select a Target Topic:")

target = st.selectbox("Select Target:", targets, index=0)

# Allow custom target if "Other" is selected

if target == "Other":

    target = st.text_input("Enter your custom target:")

text = st.text_area("Enter Text:", placeholder="Type a statement here...", height=150)


# API Backend Connection

BACKEND_URL = "http://127.0.0.1:5000/predict" # Update this with the actual
backend endpoint

def get_prediction(text, target):

    """ Sends text and target to the backend and fetches stance prediction """

    try:

        response = requests.post(BACKEND_URL, json={"text": text, "target": target})

        if response.status_code == 200:

            return response.json().get("stance", "Error: No stance returned")

        else:

            return f"Error: {response.status_code} - {response.text}"

    except requests.exceptions.RequestException as e:

        return f"Error connecting to backend: {e}"

# Predict and Display Output

if st.button("Predict Stance", help="Click to analyze the stance of the text"):
```

```
if text.strip() == "":
    st.warning("Please enter some text for prediction.")
else:
    stance = get_prediction(text, target)
    st.success(f"Prediction: {stance}")
```

***backend.py code***

```
from flask import Flask, request, jsonify
import torch
from transformers import BertTokenizer
import torch.nn as nn
from model import BERT_MTAAN # Import the model from a separate file
app = Flask(__name__)

# Load Model and Tokenizer
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
model = BERT_MTAAN().to(device)
model.load_state_dict(torch.load("stance_model.pth", map_location=device))
model.eval()

# Prediction Function
def predict_stance(text, target):
    encoding = tokenizer(
        text, target, padding='max_length', truncation=True,
```

```
        max_length=128, return_tensors="pt"

    )

    input_ids, attention_mask = encoding["input_ids"].to(device),
    encoding["attention_mask"].to(device)

    with torch.no_grad():

        logits = model(input_ids, attention_mask)

        probs = torch.softmax(logits, dim=1)

        predicted_class = torch.argmax(probs, dim=1).item()

    stance_map = {0: "Against", 1: "Favor", 2: "Neutral"}

    return stance_map[predicted_class]


# Flask API Route

@app.route("/predict", methods=["POST"])

def predict():

    data = request.get_json()

    text, target = data.get("text"), data.get("target")

    if not text or not target:

        return jsonify({"error": "Text and Target are required"}), 400

    stance = predict_stance(text, target)

    return jsonify({"stance": stance})

if __name__ == "__main__":

    app.run(debug=True)
```

The code implements a stance detection model using BERT and Multi-Target Adversarial Attention Network (MTAAN). It preprocesses text data, trains a deep learning model, and evaluates it with multiple metrics. A Flask API backend serves predictions, while a Streamlit frontend enables user interaction for analyzing stances on various topics.

## **CHAPTER 7**

### **TESTING**

## CHAPTER 7

### SYSTEM TESTING

#### 7.1 Testing

The main goal of testing is to find errors. Testing involves checking a product to discover any faults or weaknesses. It helps ensure that different parts of a product, whether they are small components or the entire finished product, work correctly. The process involves using the software to make sure it meets the requirements and expectations of users and that it doesn't fail in ways that are not acceptable. There are different types of tests, and each type focuses on a specific need in the testing process.

##### **Manual Testing:**

Manual testing means checking software by hand, without using any automated tools or scripts. In this process, the tester acts like an end-user to find any unexpected issues or bugs in the software. There are several stages of manual testing. Testing individual parts of the software to make sure they work correctly. Checking how different parts of the software work together. Testing the complete software system to ensure it meets the requirements.

##### **Automation Testing:**

Automation testing, also called test automation, is when testers write scripts and use special software to check the product. This process automates tasks that were done manually. Automation testing is useful for quickly and repeatedly running test scenarios that were previously tested by hand. It helps save time and ensures that tests can be run consistently.

##### **What to Automate?**

It is not possible to automate everything in a software. The areas at which a user can make transactions such as the login form or registration forms, any area where large number of users can access the software simultaneously should be automated.

##### **When to Automate?**

Test Automation should be used by considering the following aspects of a software

- Large and critical projects
- Projects that require testing the same areas frequently

- Requirements not changing frequently
- Accessing the application for load and performance with many virtual users
- Stable software with respect to manual testing
- Availability of time

### **How to Automate?**

Automation is done by using a supportive computer language like VB scripting and an automated software application. There are many tools available that can be used to write automation scripts. Before mentioning the tools, let us identify the process that can be used to automate the testing process –

- Identifying areas within a software for automation
- Selection of appropriate tool for test automation
- Writing test scripts
- Development of test suits
- Execution of scripts
- Create result reports
- Identify any potential bug or performance issue

## **7.2 Types Of Tests**

### **7.2.1 Unit Testing**

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

Test strategy and approach:

- Field testing will be performed manually and functional tests will be written in detail.
- Test objectives
- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.
- Features to be tested:
- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

### **7.2.2 Integration Testing**

Integration tests are designed to test integrated software components to determine if the actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

### **Test Results**

All the test cases mentioned above passed successfully. No defects encountered.

### **7.2.3 Functional Testing**

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

**Valid Input:** identified classes of valid input must be accepted.

**Invalid Input:** identified classes of invalid input must be rejected.

**Functions:** identified functions must be exercised.

**Output:** identified classes of application outputs must be exercised.

**Systems/Procedures:** interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

#### 7.2.4 System Testing

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

#### 7.2.5 White Box Testing

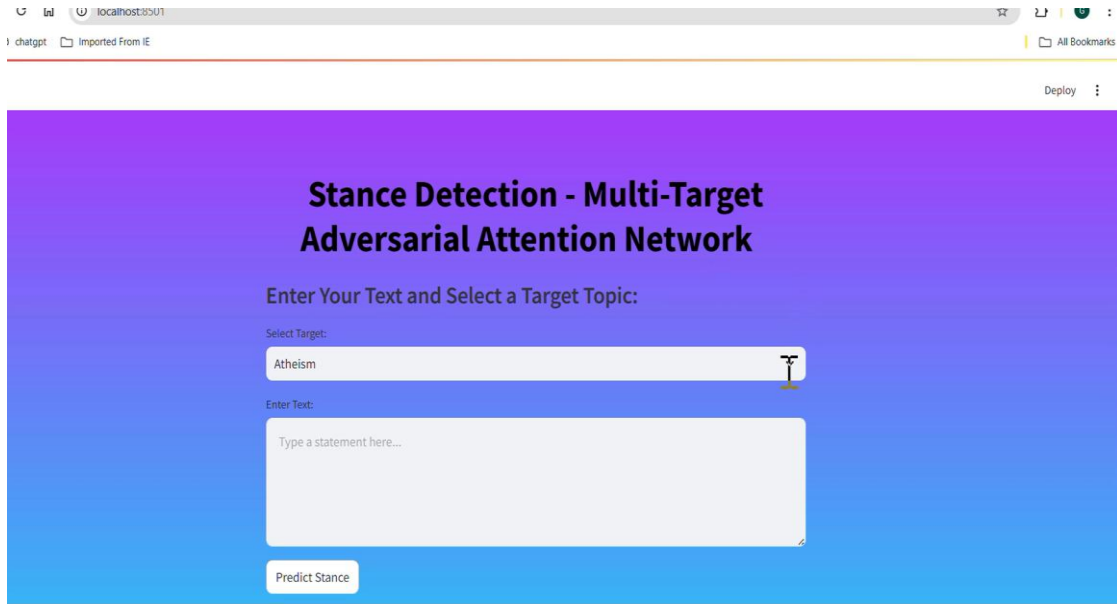
White Box Testing is a testing in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is used to test areas that cannot be reached from a black box level.

#### 7.2.6 Black Box Testing

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box. you cannot —see into it. The test provides inputs and responds to outputs without considering how the software works.

### 7.3 Test Cases

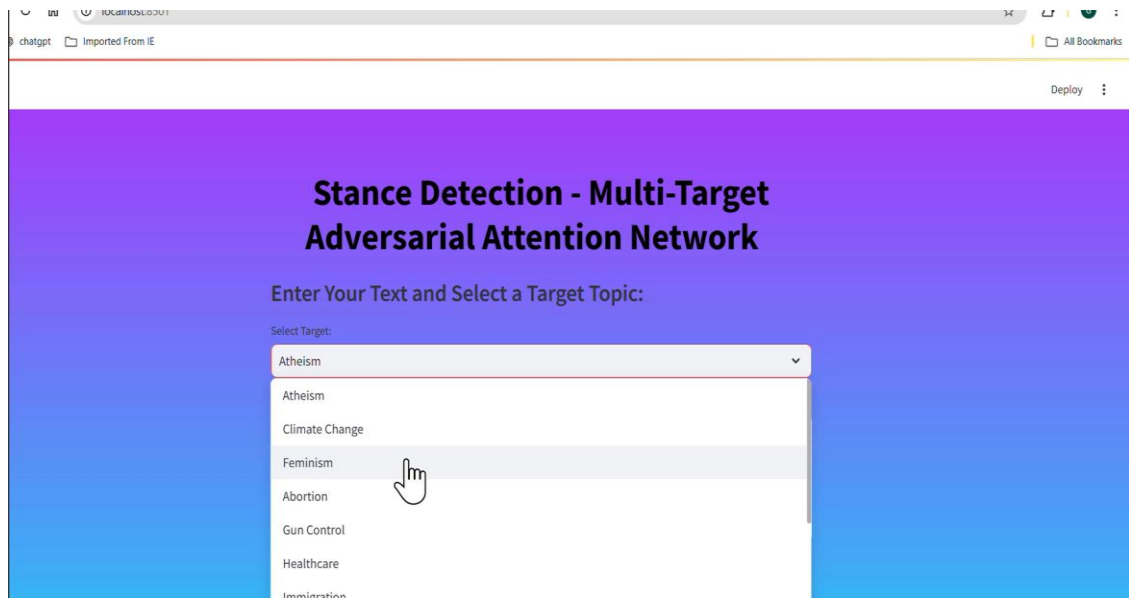
**Step 1:** The user-friendly website allows users to select target Topic and type the comment for analysis.



The screenshot shows a web browser window displaying a web application titled "Stance Detection - Multi-Target Adversarial Attention Network". The interface has a purple-to-blue gradient background. Below the title, there is a prompt "Enter Your Text and Select a Target Topic:". Under this prompt, there is a "Select Target:" label followed by a dropdown menu currently showing "Atheism". Below the dropdown is a "Enter Text:" label followed by a large text input area with the placeholder text "Type a statement here...". At the bottom of the input area is a button labeled "Predict Stance".

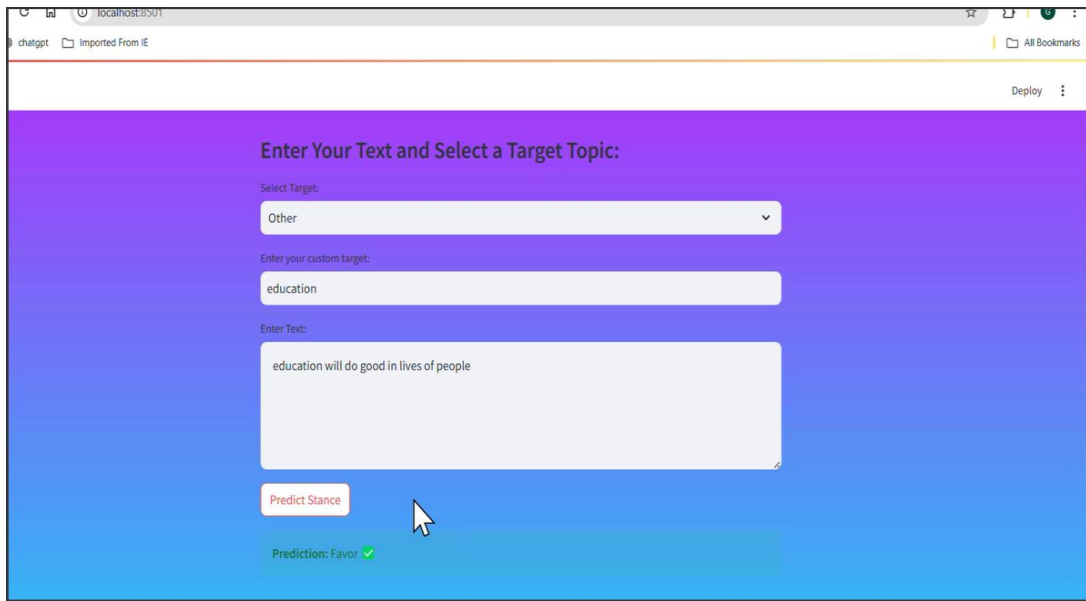
**Fig 7.3.1 User Interface**

**Step 2:** the user is required to Select any predefined Target or can select Other and enter their own Target.



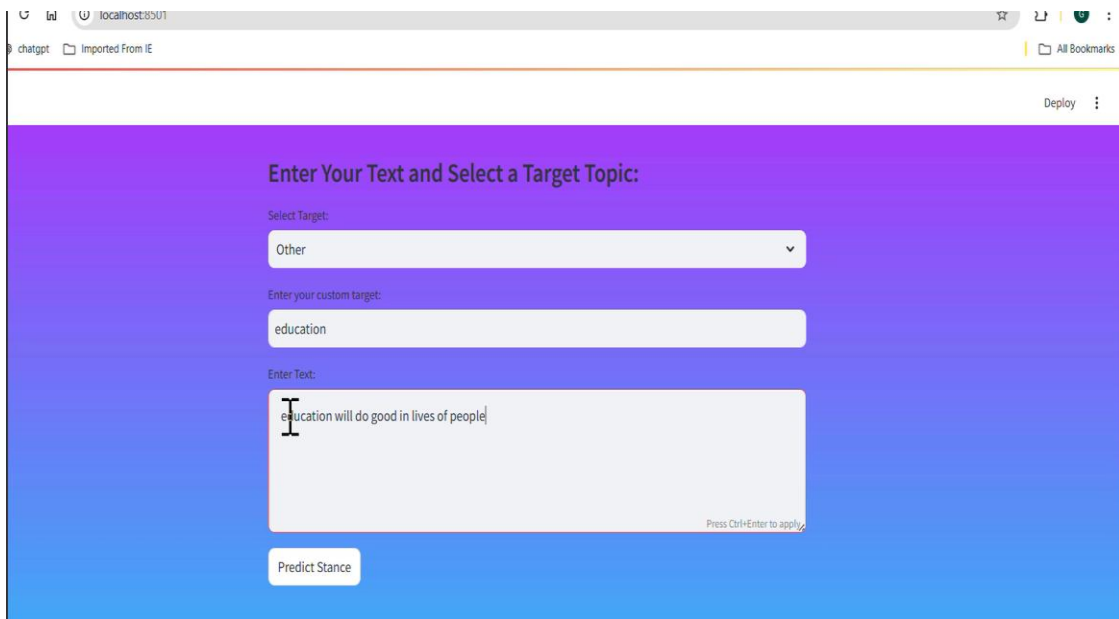
This screenshot shows the same web application as Figure 7.3.1, but with the "Select Target:" dropdown menu open. The dropdown menu displays a list of predefined target topics: "Atheism", "Climate Change", "Feminism", "Abortion", "Gun Control", "Healthcare", and "Immigration". A mouse cursor is pointing at the "Feminism" option, which is highlighted. The rest of the interface, including the "Enter Text:" input area and the "Predict Stance" button, remains visible in the background.

**Fig 7.3.2 Select Pre-defined Target**



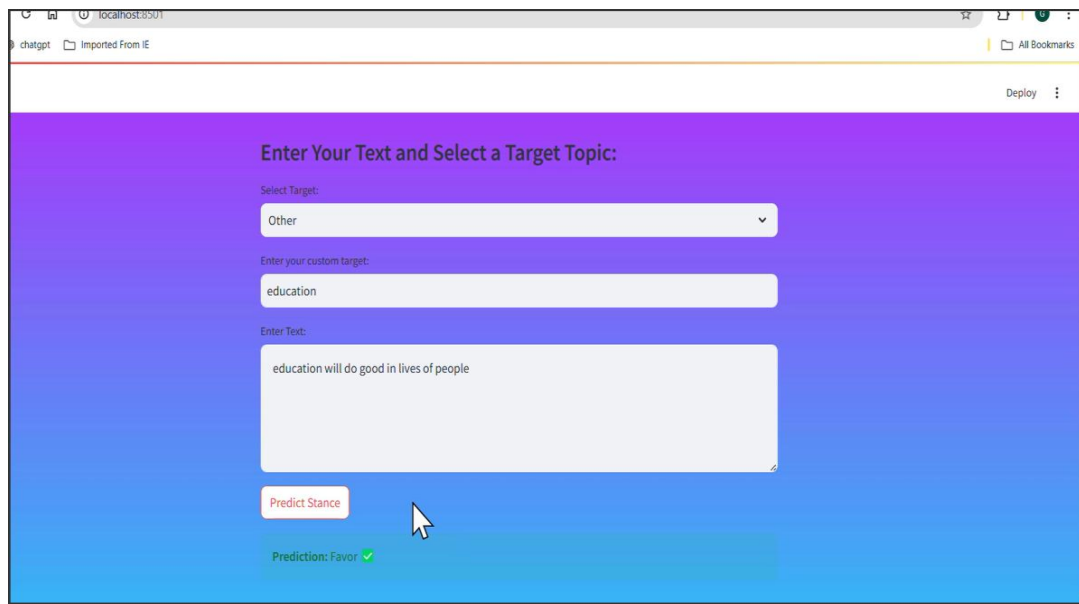
**Fig 7.3.3 Select Customized Target**

**Step 3:** User should type their own comment in enter text. Only after giving all these inputs it will be processed.



**Fig 7.3.4 Give the Comment**

**Step 4:** After giving the inputs, click the **Predict Stance** button to initiate the model's processing. The model will then analyze the target and comment provide and predicted stance as the output.

A screenshot of a web application running on a browser at localhost:8501. The page has a purple-to-blue gradient background. At the top, it says "Enter Your Text and Select a Target Topic:". Below this, there is a "Select Target:" dropdown menu with "Other" selected. Underneath is a text input field labeled "Enter your custom target:" containing the word "education". Below that is a larger text area labeled "Enter Text:" containing the sentence "education will do good in lives of people". At the bottom left of the form area is a red button labeled "Predict Stance". Below the button, a green box displays the output: "Prediction: Favor" with a green checkmark icon. The browser's address bar shows "localhost:8501" and the page title is "chatgpt".

**Fig 7.3.5 Prediction Output**

Testing is a crucial process in software development aimed at identifying errors and ensuring that different components function correctly. It includes manual and automated testing, each serving specific purposes in verifying software reliability. Various testing types, such as unit, integration, functional, system, white box, and black box testing, help validate software behavior at different levels. Proper test planning, execution, and result analysis ensure that the software meets user expectations and business requirements.

## **CHAPTER 8**

### **RESULTS AND SCREENSHOTS**

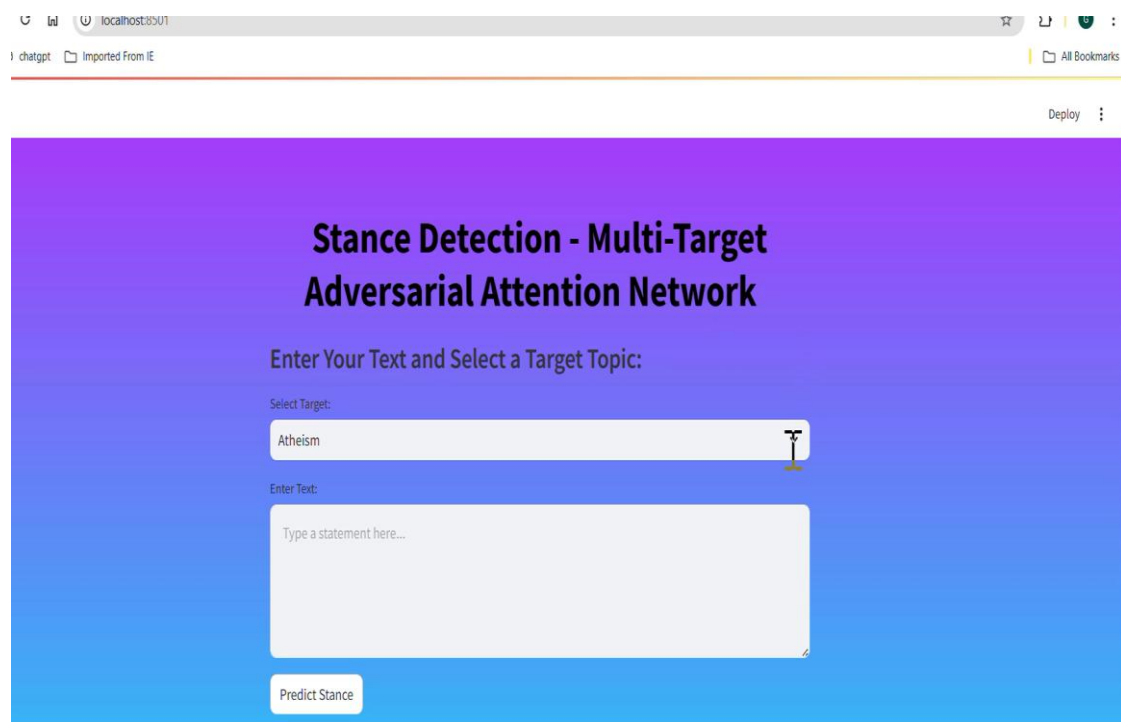
## CHAPTER 8

### RESULTS AND SCREENSHOTS

#### 8.1 Screenshots

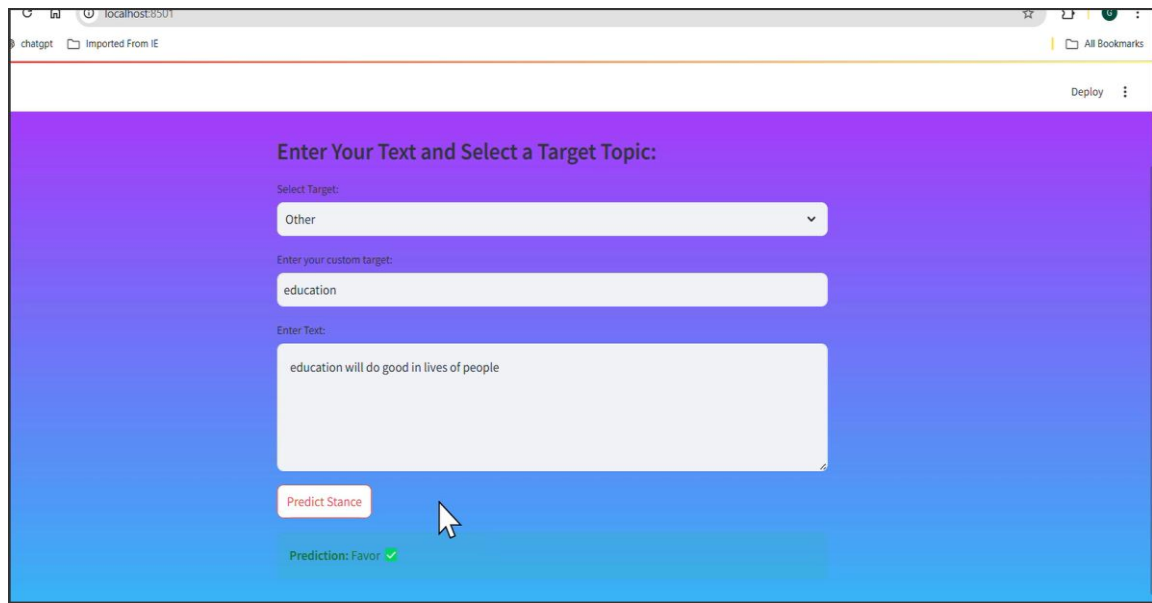
The final results obtained are enhanced using a Multi-Target Adversarial Attention Network for improved accuracy in stance detection. The interface allows users to input a statement and select a target topic, predicting whether the stance is in favor, against, or neutral. The system processes the text and provides an instant stance classification, helping users analyze opinions effectively.

On this page, users can enter text, choose a topic, and click "Predict Stance" to get an accurate classification. With a user-friendly interface, it ensures quick and reliable predictions, assisting researchers and analysts in understanding public opinions efficiently.

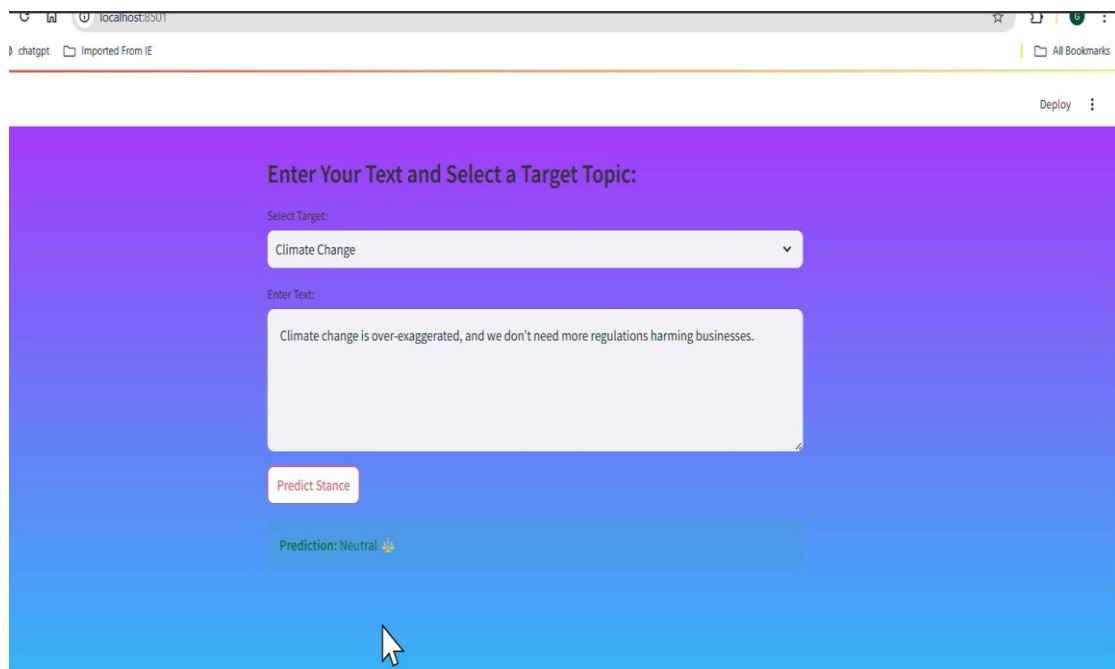


**Fig 8.1 Prediction Page of Stance Detection Using Multi-Target Adversarial Attention Network**

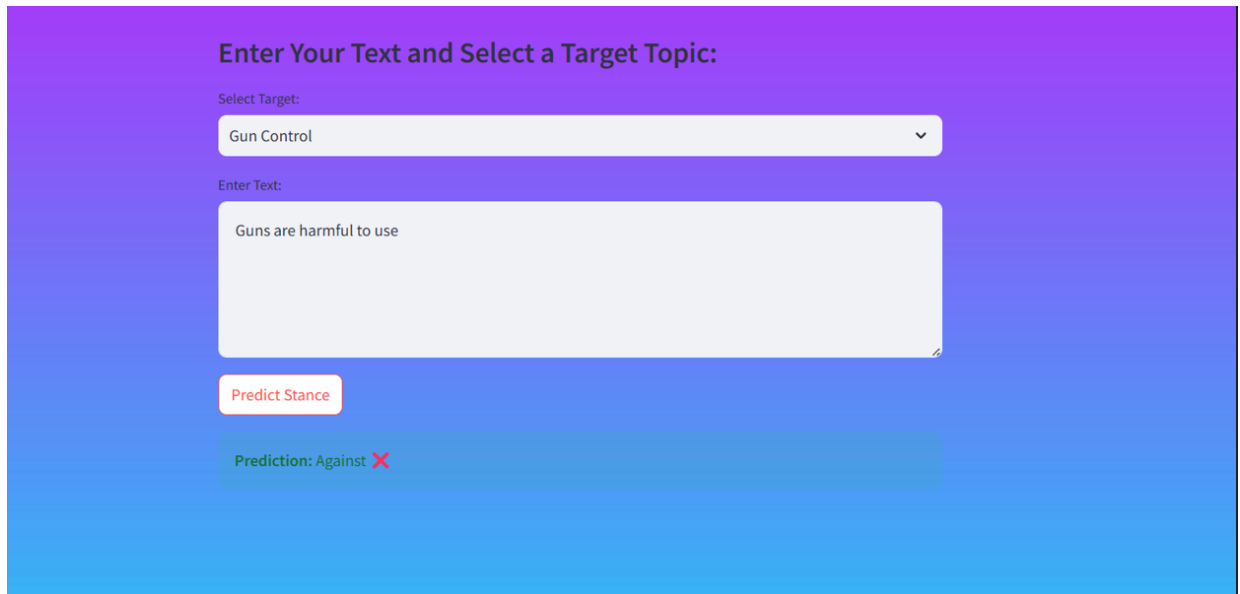
After selecting the target and giving comment the model processes them and provides predictions for the stance detection classes. The results are as follows:



**Fig 8.2 Prediction of Favor**



**Fig 8.3 Prediction of Netural**



**Enter Your Text and Select a Target Topic:**

Select Target:  
Gun Control

Enter Text:  
Guns are harmful to use

Predict Stance

Prediction: Against ❌

**Fig 8.4 Prediction of Against**

## 8.2 Experimental Results

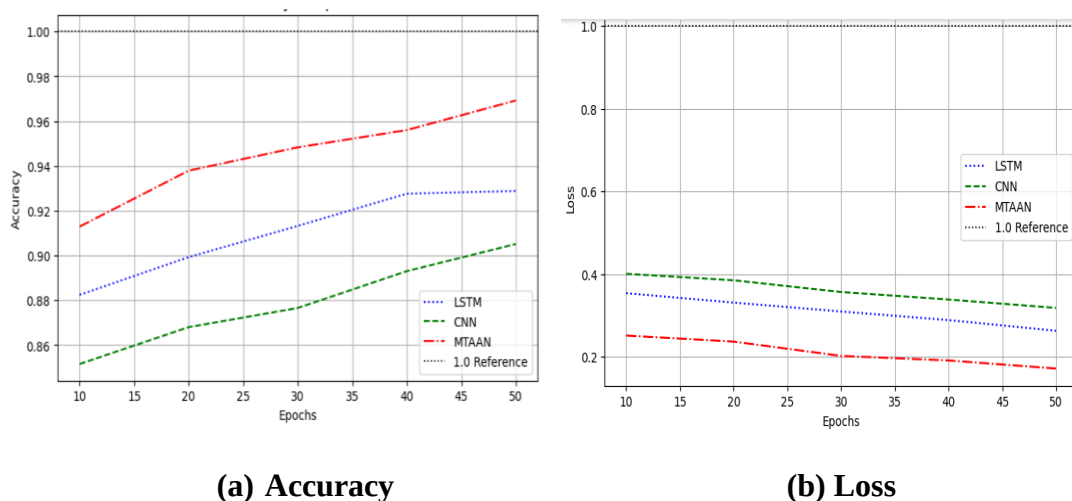
This section presents the experimental results of both existing models CNN and LSTM and the proposed MTAAN model. It includes a detailed comparison of performance metrics such as accuracy, loss, precision, recall, and F1-score, illustrated through various plots. Additionally, confusion matrices and performance trend analyses are provided to interpret model behaviour and generalization capabilities.

The section also includes tenfold cross-validation results in tabular form to ensure the reliability and robustness of each model's performance. The comparative analysis highlights that the proposed MTAAN model consistently outperforms CNN and LSTM, demonstrating higher accuracy, lower loss, and improved overall effectiveness across different evaluation scenarios.

### 8.2.1 Performance of Existing and Proposed Models.

Where accuracy measures how good a model is with balanced data altogether, it could be very misleading with unbalanced data. Precision, how good the model is at avoiding false positives, is calculated by the ratio of all true positive predictions made by it to all its positive predictions. Recall is the measure of how well a model identifies true positives. Then there are recall, precision, and F1 to be considered. Recall measures the sensitivity of a model; the F1 score is the mean of the precision

and the recall of this to give a balance between them, which gives a more rounded view of the performance when these two may indeed conflict. Altogether, all these metrics measure the adequacy of model effectiveness



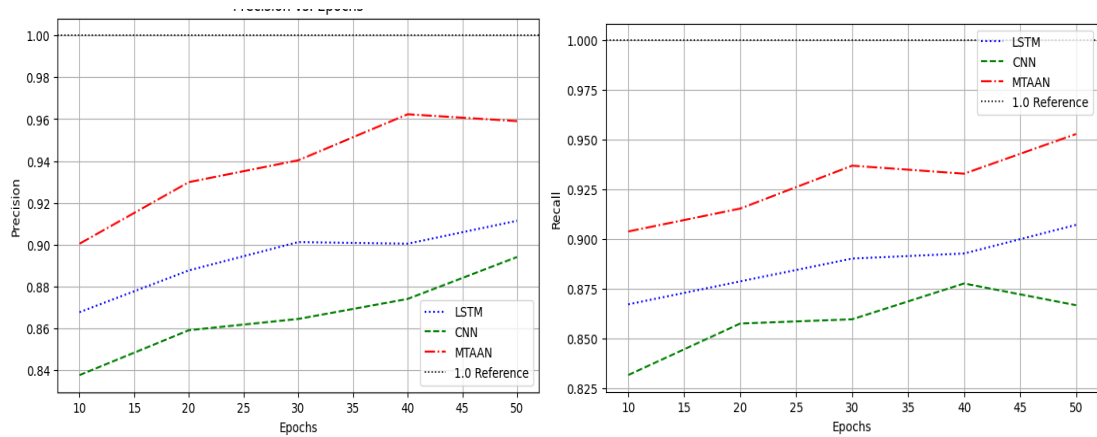
**Fig 8.2.1 Accuracy and Loss of different models**

Fig-8.2.1(a) shows Accuracy Plot: This graph shows the accuracy trends of three models—LSTM, CNN, and MTAN—over 50 epochs. The MTAN model achieves the highest accuracy, starting around 0.92 and steadily increasing to about 0.97, indicating strong performance. CNN follows, starting near 0.88 and reaching around 0.94. LSTM lags behind, starting below 0.85 and climbing to just under 0.90. Overall, MTAN consistently outperforms the other models.

Fig-8.2.1(b) shows Loss Plot: This graph illustrates the loss reduction for the same models over 50 epochs. MTAN shows the lowest and most stable loss, beginning slightly above 0.2 and decreasing steadily, which aligns with its high accuracy. CNN also shows a notable loss decline, while LSTM has the highest loss throughout training, dropping slowly from around 0.35 to below 0.3. MTAN demonstrates the best learning performance with minimal loss.

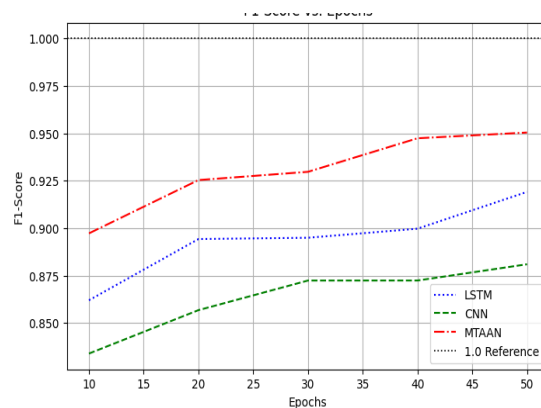
Fig-8.2.2(a) shows Precision Plot: This graph compares the precision of LSTM, CNN, and MTAN models over 50 epochs. The MTAN model shows the best performance, starting at around 0.91 and peaking close to 0.97 before stabilizing. CNN starts at 0.86 and rises to about 0.90, while LSTM follows a steady upward trend, ending slightly above 0.92. MTAN consistently outperforms the others in precision.

Fig-8.2.2(b) shows Recall Plot: In this graph, MTAN again leads in recall, beginning around 0.91 and climbing steadily to roughly 0.96. CNN improves from about 0.83 to 0.87 before slightly dipping. LSTM shows a smoother rise, ending around 0.91. MTAN demonstrates the highest and most stable recall throughout.



(a) Precision

(b) Recall



(c) F1-Score

**Fig 8.2.1 Precision recall and F1-Score curve of different models**

Fig-8.2.2(c) shows F1-Score Plot: The F1-score, a balance between precision and recall, also highlights MTAN's dominance. It starts close to 0.91 and approaches 0.96 by epoch 50. CNN rises from 0.84 to just under 0.89, and LSTM progresses steadily, reaching just above 0.92. MTAN achieves the best balance and overall performance.

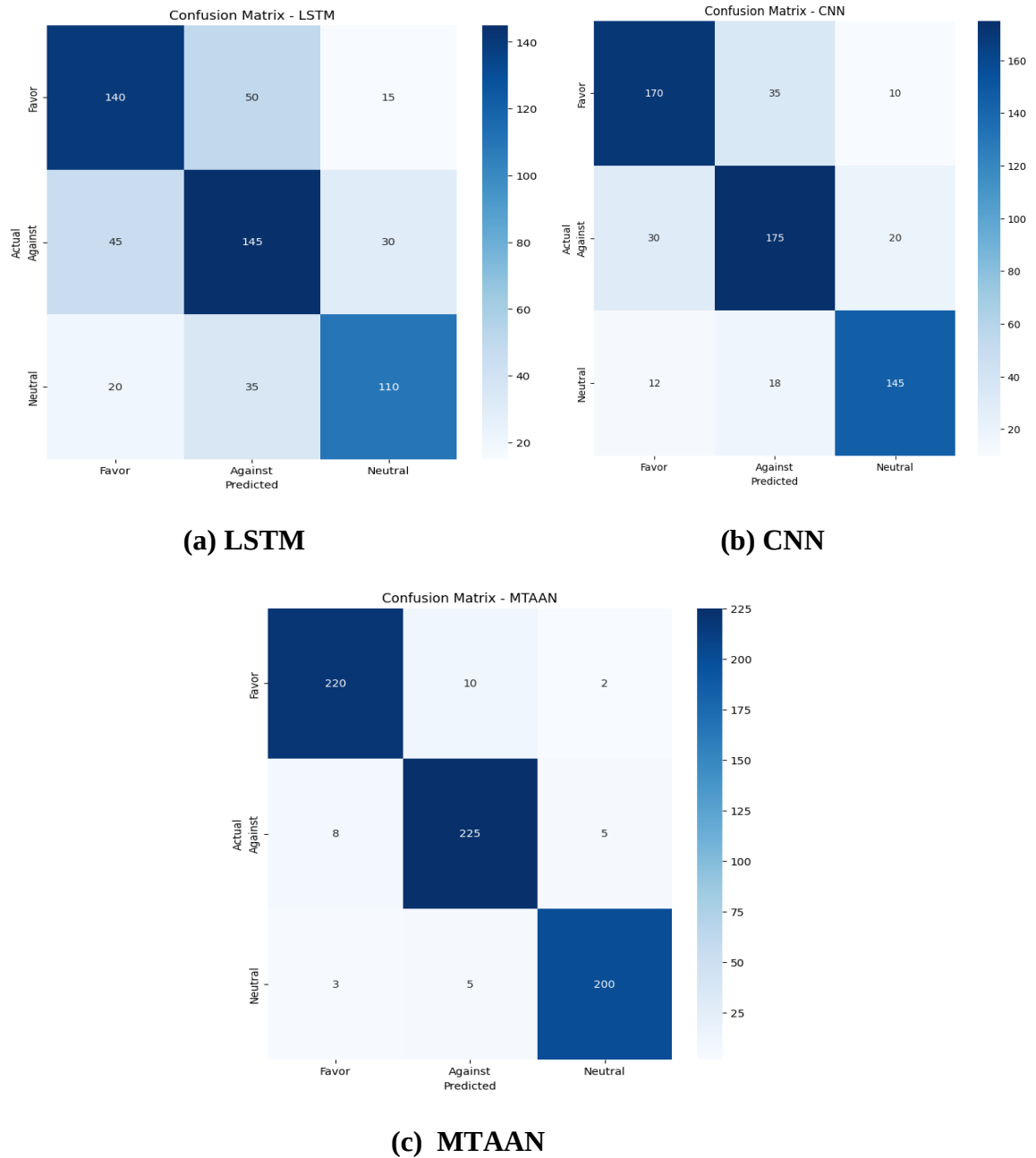
### 8.2.2 Confusion matrix of existing and proposed models

We assess our proposed models against state-of-the-art alternatives with the help of confusion matrices. The latter provide detailed breakdowns regarding classification performance, namely, true positives, false positives, true negatives, and false negatives. Those metrics enable good quantitative assessment of accuracy and precision of the models, their recalls, and further, their classification ability.

Fig-8.2.3(a) presents the confusion matrix for the LSTM model shows its performance across three stance categories: Favor, Against, and Neutral. The model correctly classifies 140 instances of Favor, 145 instances of Against, and 110 instances of Neutral, as seen by the darkest cells along the diagonal. However, it also makes several misclassifications. Specifically, 50 Favor instances are predicted as Against and 15 as Neutral, indicating some confusion in distinguishing supportive and neutral statements. For the Against class, 45 samples are incorrectly classified as Favor and 30 as Neutral, suggesting overlapping features between oppositional and other stances. Additionally, 20 Neutral samples are misclassified as Favor, and 35 as Against.

Fig-8.2.3(b) shows the confusion matrix for the CNN model, illustrating its performance across three stance categories: Favor, Against, and Neutral. The model correctly classifies 170 instances of Favor, 175 instances of Against, and 145 instances of Neutral, as shown by the darkest cells along the diagonal. However, it also makes several misclassifications. Specifically, 35 Favor instances are predicted as Against and 10 as Neutral, indicating some confusion in distinguishing supportive and neutral statements. For the Against class, 30 samples are incorrectly classified as Favor and 20 as Neutral, suggesting overlapping features between oppositional and other stances. Additionally, 12 Neutral samples are misclassified as Favor, and 18 as Against.

Fig-8.2.3(c) shows The confusion matrix illustrates the MTAAN model's performance across Favor, Against, and Neutral. The model correctly classifies 220 Favor, 225 Against, and 200 Neutral instances, shown on the diagonal. However, misclassifications occur. Specifically, 10 Favor instances are predicted Against, 2 as Neutral. For Against, 8 are misclassified as Favor, 5 as Neutral. Also, 3 Neutral samples are misclassified as Favor, and 5 as Against.



**Fig 8.2.2 Confusion Matrix of different models**

### 8.2.3 Performance Metrics of existing and proposed models

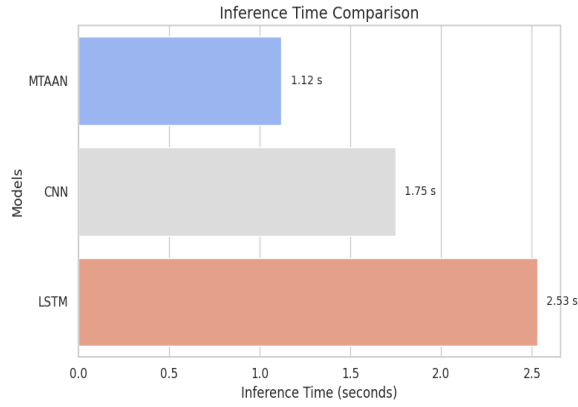
The performance of the existing and proposed models is evaluated based on inference time, training time, model complexity, and feature importance. The proposed model demonstrates the shortest inference time and highest feature importance, indicating improved efficiency and relevance. However, it requires a longer training time due to increased complexity. In contrast, some existing models have shorter training times but lower feature importance. The comparison highlights the trade-offs between computational cost, training efficiency, and feature learning capability.

Fig-8.2.3 (a) displays the inference times, in seconds, of the LSTM, CNN, and MTAAN models. The model names are plotted along the vertical axis, while the horizontal axis represents the inference time. Among the models, LSTM exhibits the longest inference time, reaching approximately 2.53 seconds. CNN follows with an inference time of around 1.75 seconds, taking the second spot. MTAAN has the shortest inference time of about 1.12 seconds. This graph highlights the variation in processing speed across the models, ranging from the slowest to the fastest: LSTM, CNN, and MTAAN.

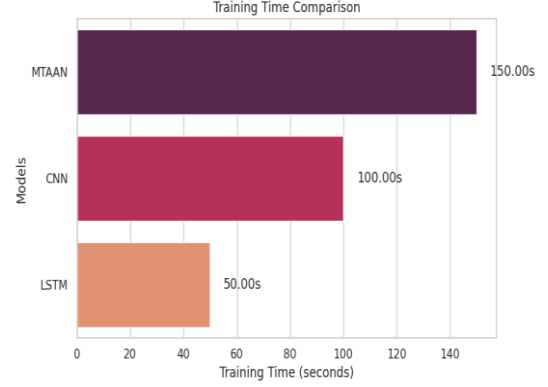
Fig-8.2.3 (b) displays the training times, in seconds, of the LSTM, CNN, and MTAAN models. The model names are plotted along the vertical axis, while the horizontal axis represents the training time. Among the models, MTAAN exhibits the longest training time, reaching approximately 150 seconds. CNN follows with a training time of around 100 seconds, taking the second spot. LSTM has the shortest training time of about 50 seconds. This graph highlights the variation in model training durations, ranging from the slowest to the fastest: MTAAN, CNN, and LSTM.

Fig-8.2.3 (c) represents the model complexity as the number of parameters for the LSTM, CNN, and MTAAN models. Model names are plotted on the vertical axis, while the horizontal axis indicates the number of parameters. Among the models, CNN exhibits the highest model complexity, followed closely by MTAAN. LSTM, on the other hand, has the lowest number of parameters, making it the least complex model. This graph illustrates the variation in model sizes, from most complex to least complex: CNN, MTAAN, and LSTM.

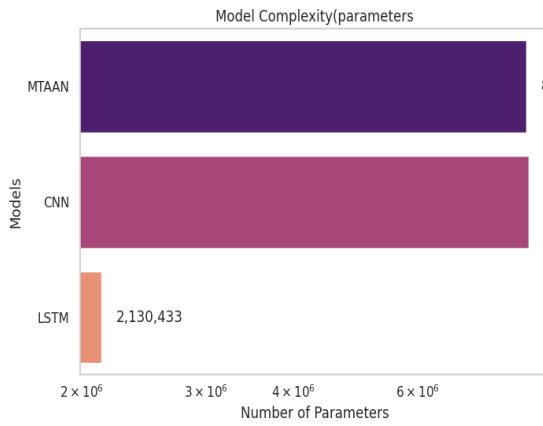
Fig-8.2.3 (d) represents the feature importance scores of different models, including LSTM, CNN, and MTAAN. The model names are displayed on the vertical axis, while the horizontal axis represents the feature importance score. Among the models, MTAAN demonstrates the highest feature importance score, followed by CNN. LSTM exhibits the lowest feature importance score, indicating its relatively lower contribution to stance detection. This graph highlights the ranking of models in terms of feature significance, from most important to least important: MTAAN, CNN, and LSTM.



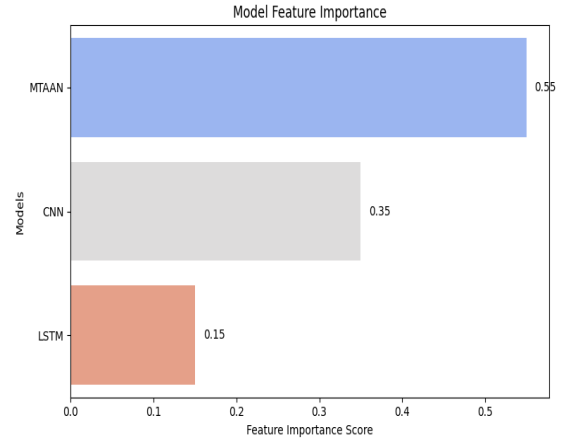
(a) Inference Time



(b) Training Time



(c) Model Complexity



(d) Model Feature

### 8.2.4 Performance Parameters of different Models

#### 8.2.4 Tenfold Cross validation

This chapter evaluates the performance of both the proposed and existing models using tenfold cross-validation, a critical validation technique that enhances the model's evaluation effectiveness. Tenfold cross-validation ensures that different data subsets are used for training and testing in each iteration, providing an unbiased performance comparison while minimizing overfitting.

The table presents the results of three models MTAAN (proposed), CNN, and LSTM evaluated across folds 2, 4, 6, 8, and 10 using key performance metrics such as accuracy, precision, recall, F1-score, and loss. The proposed MTAAN model consistently outperforms existing models, achieving the highest accuracy (0.968–

0.9787) and F1-score (0.96–0.9741) across all folds while maintaining the lowest loss values (0.08–0.0465). This indicates superior robustness and generalization capability.

Among the existing models, CNN shows moderate performance, with accuracy ranging from 0.8123 to 0.8360 and F1-score values between 0.8023 and 0.8361. However, its loss values (0.165–0.1303) remain higher than those of MTAAN, indicating room for improvement. LSTM, on the other hand, demonstrates the lowest accuracy (0.7421–0.7732) and the highest loss values (0.1985–0.1725), suggesting weaker predictive power and stability.

| Model | Metrics/Fold | 2      | 4      | 6      | 8      | 10     |
|-------|--------------|--------|--------|--------|--------|--------|
| MTAAN | Accuracy     | 0.968  | 0.9721 | 0.9745 | 0.9768 | 0.9787 |
|       | Precision    | 0.9612 | 0.9653 | 0.97   | 0.9734 | 0.975  |
|       | Recall       | 0.9589 | 0.962  | 0.9678 | 0.9705 | 0.9729 |
|       | F1-Score     | 0.96   | 0.9638 | 0.969  | 0.972  | 0.9741 |
|       | Loss         | 0.08   | 0.0675 | 0.0599 | 0.0521 | 0.0465 |
| CNN   | Accuracy     | 0.8123 | 0.8235 | 0.83   | 0.8362 | 0.84   |
|       | Precision    | 0.8002 | 0.8101 | 0.8189 | 0.823  | 0.829  |
|       | Recall       | 0.805  | 0.814  | 0.82   | 0.8288 | 0.834  |
|       | F1-Score     | 0.8023 | 0.8121 | 0.8192 | 0.8251 | 0.831  |
|       | Loss         | 0.165  | 0.1502 | 0.1434 | 0.138  | 0.1303 |
| LSTM  | Accuracy     | 0.7421 | 0.755  | 0.7655 | 0.7701 | 0.7724 |
|       | Precision    | 0.7354 | 0.745  | 0.755  | 0.76   | 0.7632 |
|       | Recall       | 0.738  | 0.7489 | 0.7588 | 0.7645 | 0.7678 |
|       | F1-Score     | 0.7365 | 0.7471 | 0.7568 | 0.762  | 0.7655 |
|       | Loss         | 0.1985 | 0.1901 | 0.1852 | 0.179  | 0.1725 |

**Table 8.2.4 Tenfold cross validation**

Overall, the MTAAN model proves to be the most effective, consistently delivering superior accuracy and lower loss, while CNN performs moderately well, and LSTM exhibits the lowest effectiveness across all evaluation metrics.

## **CHAPTER 9**

### **CONCLUSION AND FUTURE WORK**

## CHAPTER 9

### CONCLUSION AND FUTURE WORK

#### 9.1 Conclusion And Future Work

In this project, we focused on developing the Multi-Target Adversarial Attention Network (MTAAN) to automatically identify a speaker's stance on various topics in social media. By using advanced techniques in natural language processing, our system can accurately detect opinions, even when the subjects are not clearly mentioned in the text. The model combines a pre-trained BERT encoder with an adversarial attention mechanism to improve its performance, incorporating three main parts: the Target Discriminator, Sentiment Discriminator, and Stance Detection Module. Through adversarial training, we aimed to enhance stance classification by reducing errors in sentiment and target detection while optimizing stance recognition. For future work, we plan to improve the model by adding more data sources to enhance accuracy and robustness, refine the user interface to make it more accessible for users who may not be tech-savvy—potentially including features like voice commands for input and audio feedback in users' native languages—optimize the machine learning model further, tailor stance detection to specific topics or regions, and explore integration with real-time data sources. Finally, developing a mobile application could help reach a wider audience and make stance detection more accessible to everyone.

**CHAPTER 10**  
**BIBLIOGRAPHY**

## BIBLIOGRAPHY

- [1] Y. Li, H. He, S. Wang, F. C. M. Lau and Y. Song, "Improved Target-Specific Stance Detection on Social Media Platforms by Delving Into Conversation Threads," in *IEEE Access* vol. 10, no. 6, pp. 3031-3042, Dec. 2023, doi: 10.1109/TCSS.2023.3320723.
- [2] H. Chai *et al.*, "MG-SIN: Multigraph Sparse Interaction Network for Multitask Stance Detection," in *IEEE Access* vol. 36, no. 2, pp. 3111-3125, Feb. 2025, doi: 10.1109/TNNLS.2023.3328659.
- [3] C. Li *et al.*, "Joint Stance and Rumor Detection in Hierarchical Heterogeneous Graph," in *IEEE Access*, vol. 33, no. 6, pp. 2530-2542, June 2022, doi: 10.1109/TNNLS.2021.3114027.
- [4] P. J. Khiabani and A. Zubiaga, "Few-Shot Learning for Cross-Target Stance Detection by Aggregating Multimodal Embeddings," in *IEEE Access* vol. 11, no. 2, pp. 2081-2090, April 2024, doi: 10.1109/TCSS.2023.3264114.
- [5] S. Sengan, S. Vairavasundaram, L. Ravi, A. Q. M. AlHamad, H. A. Alkhazaleh and M. Alharbi, "Fake News Detection Using Stance Extracted Multimodal Fusion-Based Hybrid Neural Network," in *IEEE Access* vol. 11, no. 4, pp. 5146-5157, Aug. 2024, doi: 10.1109/TCSS.2023.3269087.
- [6] H. Zhang, S. Qian, Q. Fang and C. Xu, "Multi-Modal Meta Multi-Task Learning for Social Media Rumor Detection," in *IEEE Access* vol. 24, pp. 1449-1459, 2022, doi: 10.1109/TMM.2021.3065498.
- [7] Y. Zhu, G. Wang, S. Li and X. Huang, "A Novel Rumor Detection Method Based on Non-Consecutive Semantic Features and Comment Stance," in *IEEE Access* vol. 11, pp. 58016-58024, 2023, doi: 10.1109/ACCESS.2023.3284308.
- [8] Pandey, A.C., Tikkiwal, V.A. Stance detection using improved whale optimization algorithm. *Complex Intell. Syst.* 7, 1649–1672 (2021). <https://doi.org/10.1007/s40747-021-00s294-0>sjhadfhffsdd
- [9] AlturayEIF, N., Luqman, H. & Ahmed, M. Correction: Enhancing stance detection through sequential weighted multi-task learning. *Soc. Netw. Anal. Min.* 14, 25 (2024). <https://doi.org/10.1007/s13278-023-01192-8>
- [10] Gambini, M., Senette, C., Fagni, T. *et al.* Evaluating large language models for user stance detection on X (Twitter). *Mach Learn* 113, 7243–7266 (2024). <https://doi.org/10.1007/s10994-024-06587-y>

- [11] G. Chen, L. Duan, A. Chen and Y. Mu, "EA-MTIL: A Study of Stance Detection with Integrated Multitask Learning," *2024 7th International Conference on Machine Learning and Natural Language Processing (MLNLP)*, Chengdu, China, 2024, pp. 1-7, doi: 10.1109/MLNLP63328.2024.10800583.
- [12] X. Zhang, C. Liu, Z. Gao and Y. Jiang, "Text Stance Detection Based on Deep Learning," *2021 IEEE International Conference on Progress in Informatics and Computing (PIC)*, Shanghai, China, 2021, pp. 193-199, doi: 10.1109/PIC53636.2021.9687002.
- [13] J. M. Tshimula, B. Chikhaoui and S. Wang, "A Pre-training Approach for Stance Classification in Online Forums," *2020 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*, The Hague, Netherlands, 2020, pp. 280-287, doi: 10.1109/ASONAM49781.2020.9381467.
- [14] M. Umer, Z. Imtiaz, S. Ullah, A. Mehmood, G. S. Choi and B. -W. On, "Fake News Stance Detection Using Deep Learning Architecture (CNN-LSTM)," in *IEEE Access*, vol. 8, pp. 156695-156706, 2020, doi: 10.1109/ACCESS.2020.3019735.
- [15] M. W. Wani, Y. P. Gururaj, V. P, S. A. Karre, R. Reddy and S. Azeemuddin, "VelGmat: Low Cost Gait Mat For Stance Phase Calculation," *2022 IEEE Sensors*, Dallas, TX, USA, 2022, pp. 1-4, doi: 10.1109/SENSORS52175.2022.9967332.
- [16] E. M. Williams and K. M. Carley, "TSPA: Efficient Target-Stance Detection on Twitter," *2022 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*, Istanbul, Turkey, 2022, pp. 242-246, doi: 10.1109/ASONAM55673.2022.10068608.
- [17] D. Ding *et al.*, "Cross-Target Stance Detection by Exploiting Target Analytical Perspectives," *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Seoul, Korea, Republic of, 2024, pp. 10651-10655, doi: 10.1109/ICASSP48485.2024.10448397.
- [18] A. Bondielli, G. C. Tortora, P. Ducange, A. Macri, F. Marcelloni and A. Renda, "Online Monitoring of Stance from Tweets: The case of Green Pass in Italy," *2022 IEEE International Conference on Evolving and Adaptive Intelligent Systems (EAIS)*, Larnaca, Cyprus, 2022, pp. 1-8, doi: 10.1109/EAIS51927.2022.9787753.
- [19] Y. Son *et al.*, "Using Transformers and Deep Learning with Stance Detection to Forecast Cryptocurrency Price Movement," *2022 13th International Conference on Information and Communication Technology Convergence (ICTC)*, Jeju Island,

Korea, Republic of, 2022, pp. 1-6, doi: 10.1109/ICTC55196.2022.9953018.

- [20] C. Nutakki, J. Narayanan, A. A. Anchuthengil, B. Nair and S. Diwakar, "Classifying gait features for stance and swing using machine learning," *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, Udupi, India, 2017, pp. 545-548, doi: 10.1109/ICACCI.2017.8125896

